



O'REILLY®

AI Agents Bootcamp Day 1

Foundations of Agent Design and Multi-Agent Collaboration

Nicole Koenigstein

Course Objectives: What this Course Covers

- Learn the core mental models for agent design and how they map to frameworks such as LangGraph and CrewAI
- Use LangGraph to build single-agent and multi-agent systems with typed states, tool use, memory, and conditional decision loops
- Understand and apply collaboration and communication patterns in multi-agent systems
- Learn how to transform unstructured, semi structured, and multimodal inputs into agent-ready structured data using schema-based prompting, Pydantic validation, and MCP-driven communication patterns
- Select appropriate model types and agent roles for planning, retrieval, tool use, and fast intra-agent communication
- Design memory and context strategies that support both individual agent reasoning and group-level coordination
- Apply guardrails, secure tool execution environments, and design threat-aware workflows

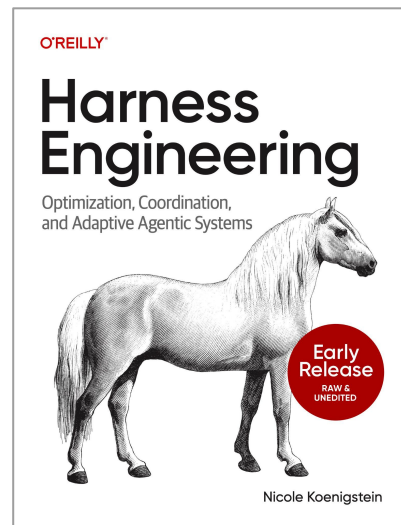
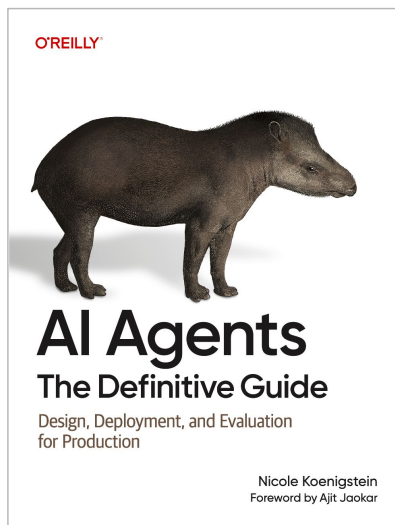
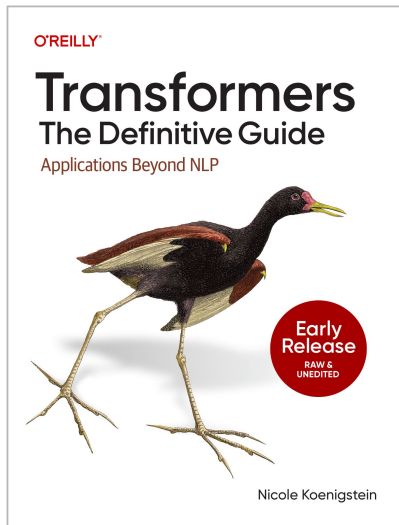
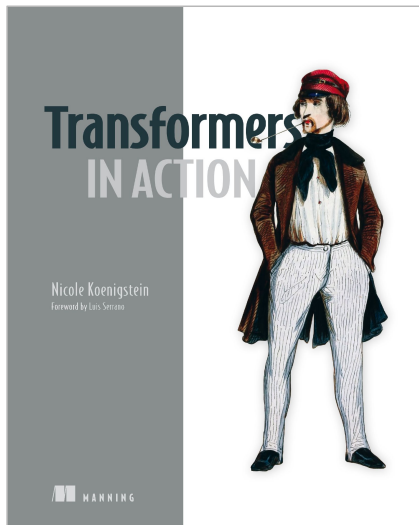
Course Goal: What You'll learn

Agents aren't magic. They are carefully engineered systems that apply structured workflows to complex problems. Which is why this course follows a high-level roadmap to guide you from ideation to production. This is framework independent, LangGraph is simply the concrete implementation vehicle.

- **Phase 1 (Control):** Start by building a simple, stateful agent with basic tool use.
- **Phase 2 (Intelligence):** Add intelligence by implementing structured reasoning (CoT, ReAct) Add Human-in-the-Loop to make the system smarter and safer.
- **Phase 3 (Coordination):** Scale up by designing a multi-agent system and by choosing the proper communication and collaboration patterns (such as Hierarchical, Swarm, etc.) to handle complexity. Choosing the right models for each task.
- **Phase 4 (Reliability):** For production focus on the core pillars, such as validation, guardrails and fallbacks, to ensure the system is reliable, auditable, and ready for deployment.

About Me

AI Researcher and Practitioner in Agentic Systems



Research, consulting, teaching, and direct system implementation focused on building reliable, production-ready agentic systems, with emphasis on multi-agent architectures, evaluation, safety, and long-term system behavior. Currently pursuing a PhD on self-improving AI agents and agent benchmarking.

Advised the European Commission and IOSCO on generative AI in regulated environments and serving on advisory boards for leading AI and quantitative finance conferences. Delivered 50+ invited talks and technical workshops across academia, industry, and international conferences in the past 2.5 years.

Get the Code & Course Solutions

GitHub Repo



GitHub for course code.

Solutions Link



Link to Accompanying book website for
AI Agents – The Definitive Guide
to download course solutions after the course.
(<https://ai-agents-the-definitive-guide.com/courses>)

Password: AI_Agents_Bootcamp_25_03_2026

Stay Connected

Book Website



Accompanying book website for
AI Agents – The Definitive Guide
with additional resources.

Book Discord



Discord channel for AI Agents
– The Definitive Guide

LinkedIn



Continue the Learning

Nicolepcx / ORM_AI_Agents_Bootcamp

Code Issues Pull requests **Discussions** Actions Projects Wiki Security Insights Settings

Category "General" has been deleted.

Manage discussion categories

Sections are a dropdown of categories. Categories have types of discussions, and discussions within them.

New section New category

Categories without section

	Announcements Updates from maintainers	 
	Ideas Share ideas for new features	 
	Q&A Answers enabled · Ask the community for help	 
	Show and tell	 

Share your ideas and projects after the course, and get your questions answered beyond the course.

Session 1

Foundations of Agent Design and Task Decomposition

Course Schedule

Session 1

Foundations of agent design and task decomposition (60 minutes)

- **Intro** (5 mins)
- **Setup the coding environment. (5 mins)**
- **Presentation:** State machines, structured state transitions, and how to translate real-world tasks into agent workflows using LangGraph
- **Demo:** Transforming a stateless LLM into a stateful agent with basic tool use
- **Hands-on Exercise:** Decompose a given small task and implement it as a LangGraph workflow with state and one tool
- **Q&A** (5 mins)



Mental Model

Architecting the Workflow



From LLMs to Agents: The Foundational Blueprint

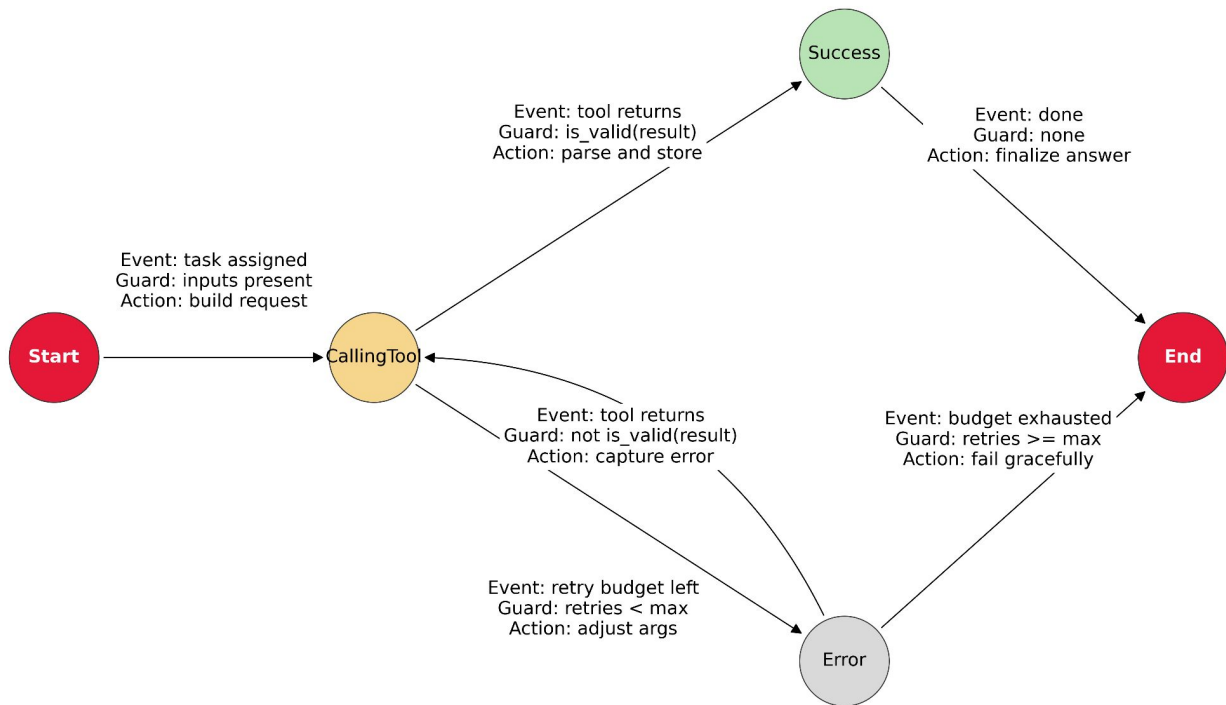
How do we move from one action to the next in a structured, repeatable and controllable way?

State Machines as a Blueprint

State Machine = A system where:

- The agent exists in a defined state
- Based on input + logic, it transitions to the next state
- Each state has an associated function or behavior

Finite State Machines



High level overview of a finite state machine for tool use.

Finite State Machines

- **State:** compact snapshot that captures what the system knows at a given moment. For an agent, this might be the message list, routing hints, or progress markers.
- **Event:** Something that happens since the last decision, such as a tool being invoked or returning a result.
- **Guard:** A check that decides which transition to take, based on the current state and latest event.
- **Action:** Work performed during a transition, like invoking a tool, appending a message, saving a checkpoint, or requesting human input.
- **Termination:** A condition that signals the process has reached the end.

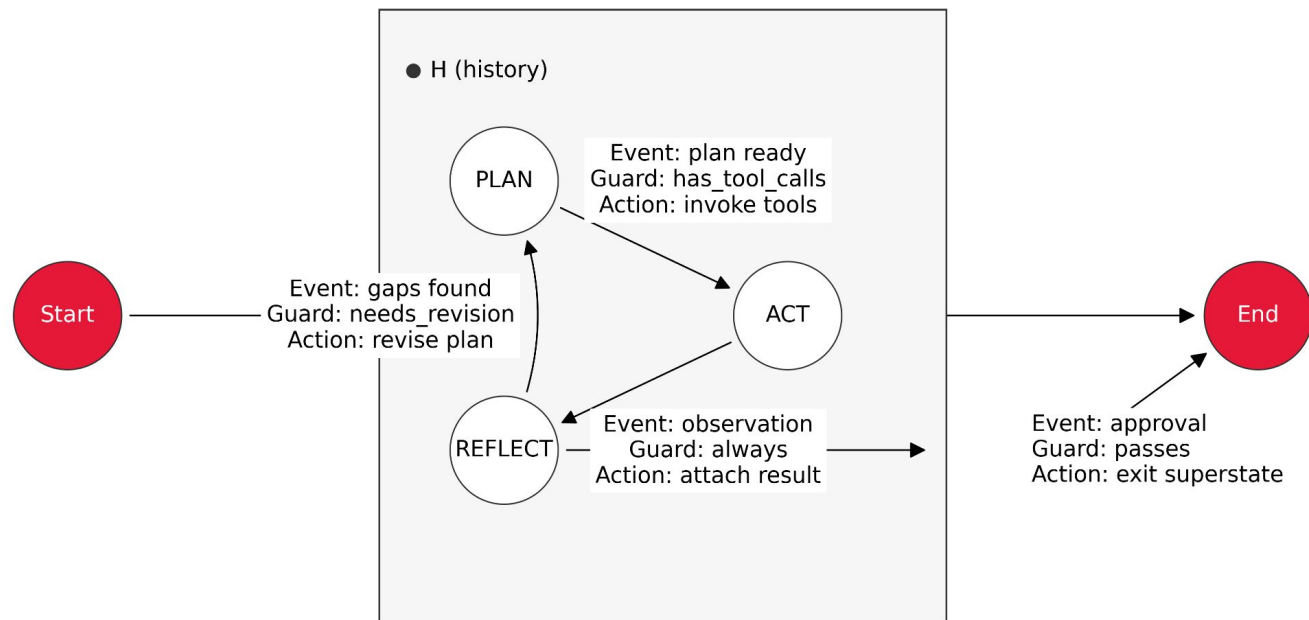
The Base Paradigm of Agents

To form a mental model of agents think of state machines which underpin modern AI agent frameworks.

They are structured transitions. An approach long used in compilers, GUIs, embedded systems, and networking protocols, but now applied to reasoning and tool use in LLM agents.



Hierarchical State Machines (HSM)

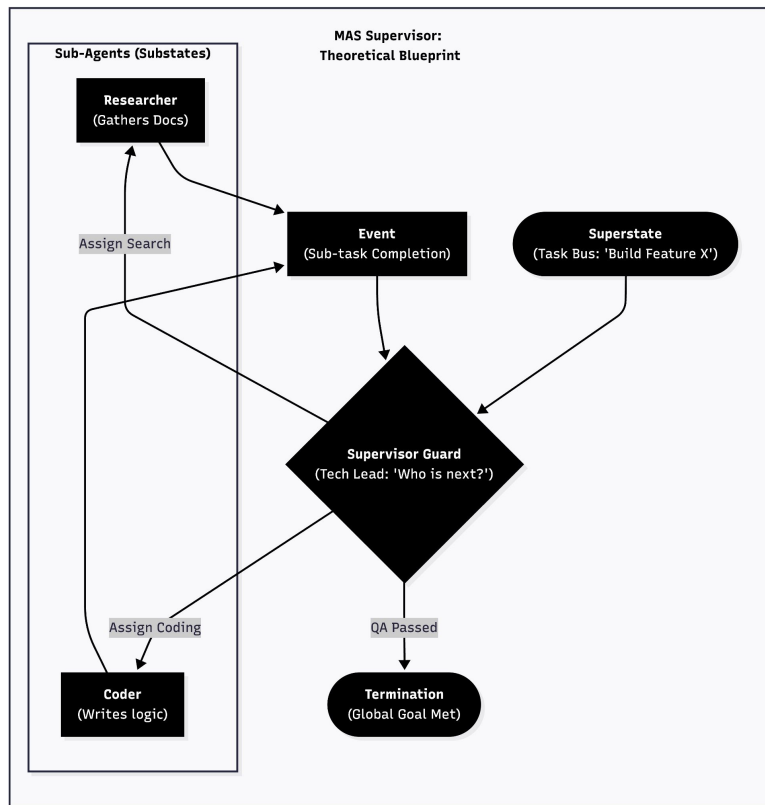


Hierarchical state machines let states nest within others. Superstates define shared rules, while substates extend them. History nodes record the last active substate, allowing an exact resume later.

Hierarchical State Machines

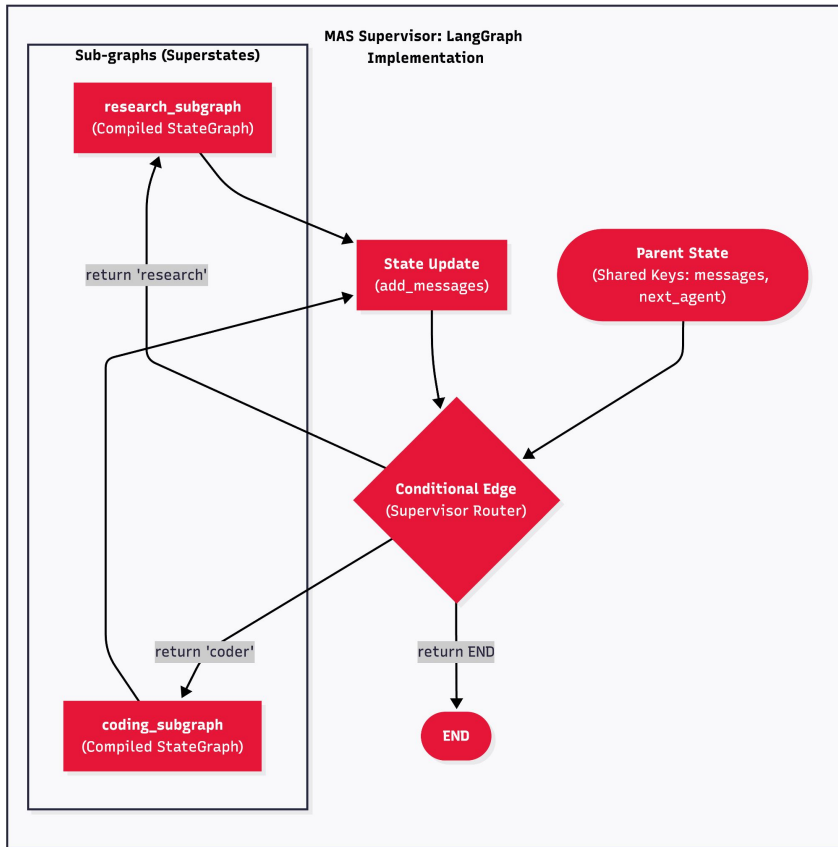
- **Superstate:** A state that groups a set of child states and their common policies. Entering the superstate runs shared logic once; exiting it runs shared cleanup once.
- **Substate:** A concrete mode that lives inside a superstate. It has its own edges but inherits the superstate's guards and actions.
- **History:** A “remember where I was” marker. Shallow history resumes at the last active child; deep history resumes inside nested grandchildren. In agent terms, this is a checkpoint for a portion of the graph.
- **Parallel region:** Two or more child regions of a superstate that advance independently. Useful for fanning out tool calls or agent roles and then joining.

Multi-Agent Orchestration (HSM)



The Supervisor acts as the governing FSM, while the specialized agents function as internal sub-circles.

Multi-Agent Orchestration (HSM)



From a Lone Contributor to a Tech Lead (the Supervisor) managing Specialized Teams (Sub-graphs).



From Stateless to Stateful

State Machines as a Blueprint (LangGraph Context)

A stateful agent uses a shared data structure to represent its current state. In LangGraph we have:

- Nodes = actions or agents
- Edges = transitions, conditional on LLM output
- State object = carries memory, inputs, and metadata

Giving the LLM a State

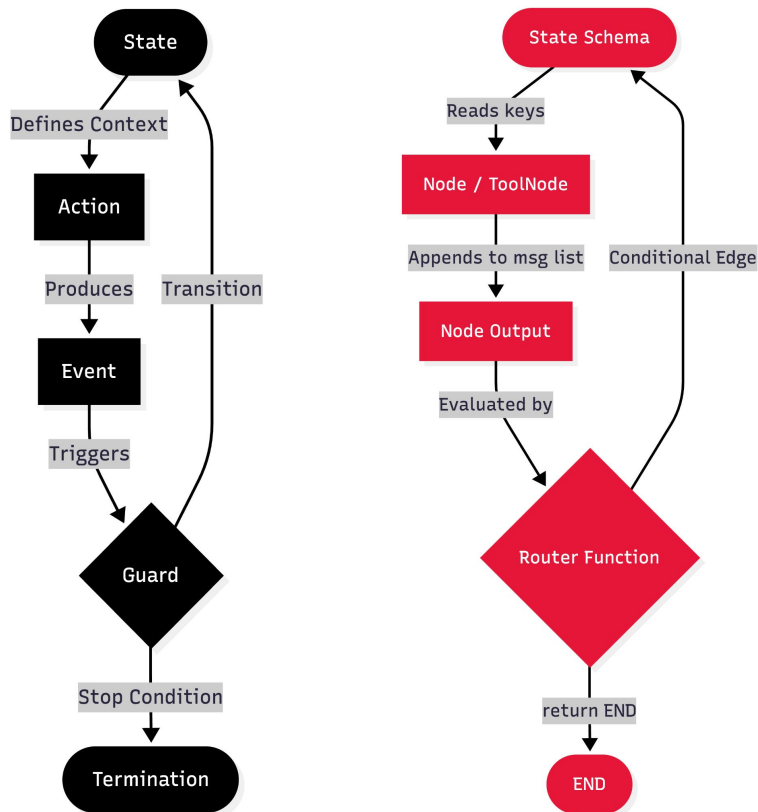
State is what makes the model act, not just respond.

Give it a structured state that tracks where it's in the workflow. Based on inputs and logic, it transitions between states, each tied to a specific behavior or function. In LangGraph, that state is typically defined as a TypedDict or Pydantic BaseModel.

This makes the system agentic:

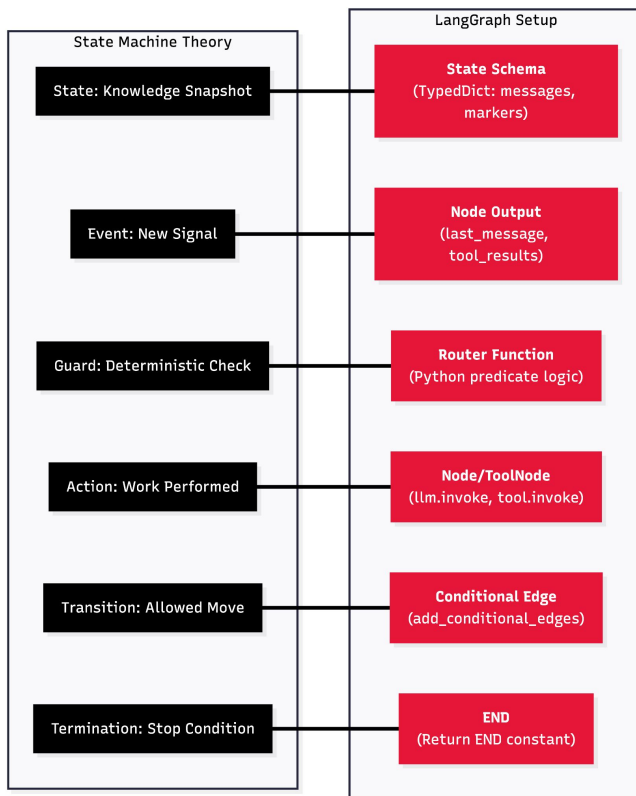
- The LLM node reasons about the next step.
- The tool node executes external actions.
- Edges route control flow based on whether tools were requested.

State Machines Mapping to LangGraph



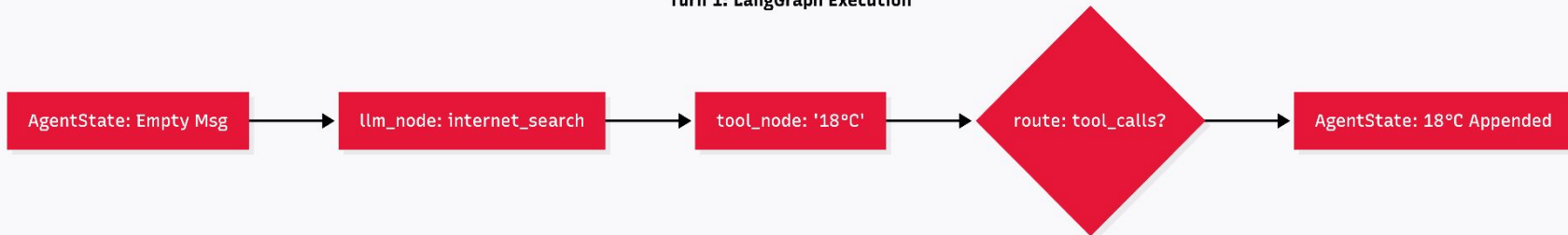
FSM vs. LangGraph flow.

Core Vocabulary Mapping: FSM to LangGraph

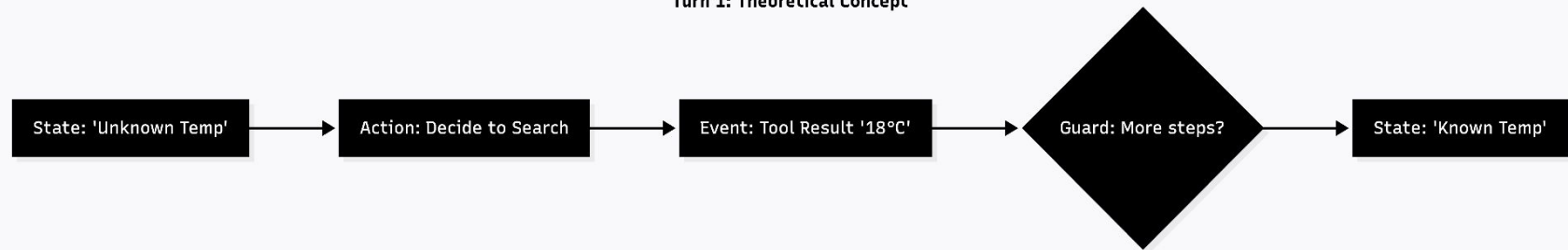


Giving the LLM a State

Turn 1: LangGraph Execution



Turn 1: Theoretical Concept





Translating Real-World Tasks



Making the Right Hire

Just as in human teams, each member has different strengths, weaknesses, and ways of working, and their skills complement one another. Same holds true with AI Agents. Think about which qualities you need for your agent to select the model.

"Hiring" your Agents the Right Way

- **Decoder-only (The Generators and Planners):** These are your standard LLMs like GPT or Claude. They are your talkers and coders, strong at drafting and step-by-step planning. Use them for execution: writing, coding, and producing candidate solutions.
- **Encoder-only (The Analysts and Investigators):** Models like BERT and its modern representations such as ModernBERT and NeoBERT do not talk, they understand. They build contextual embeddings and are excellent at semantic search, filtering, and relevance scoring. Use them to rank, verify, and narrow the search space before your expensive generator even wakes up.

"Hiring" your Agents the Right Way

- **Mixture of Experts (The Specialists):** MoE models behave like a set of internal specialist departments, where a router activates only a subset of experts per token. Use them when you need high capability, but want to spend compute selectively.
- **Reasoning Models (The Thinkers):** These are models optimized to spend more compute at test-time. They pause, reflect, and check their own reasoning. They are slower, but they often prevent expensive downstream mistakes.

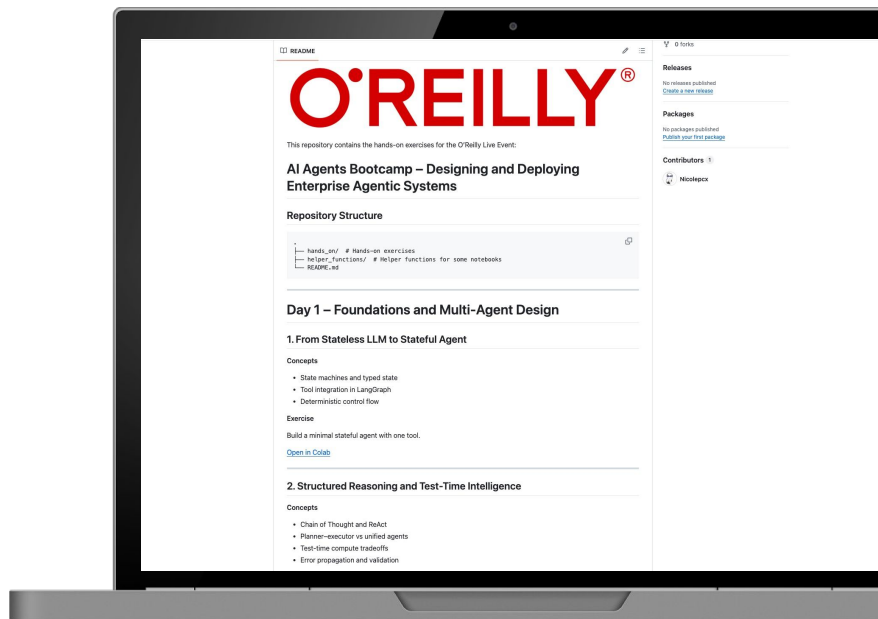
Set up Environment

GitHub Repo



GitHub for course code:

https://github.com/Nicolepcx/ORM_AI_Agents_Bootcamp





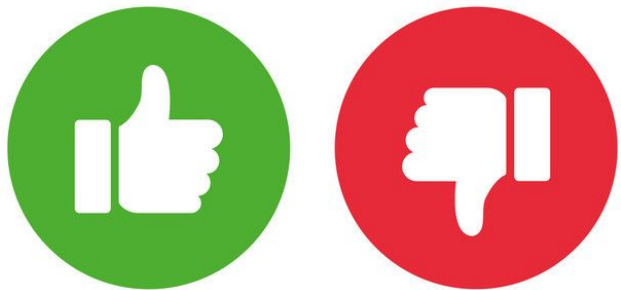
Key Takeaways Presentation Session 1

Foundations of agent design and task decomposition

- Agents are state machines
- State is the source of truth
- Tools are transitions
- Control flow must be explicit

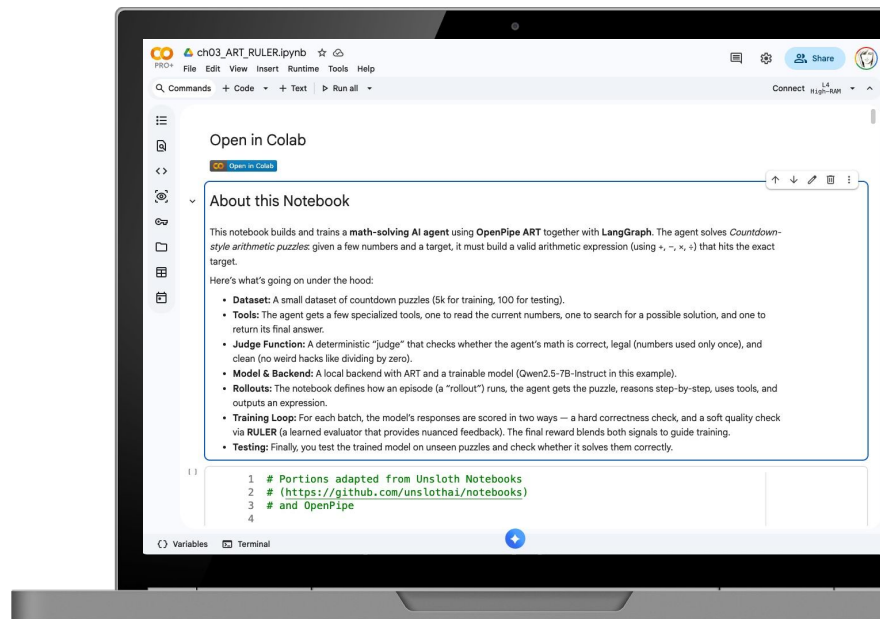
Pulse Check

Are the foundations of agent design and the link to frameworks clear?



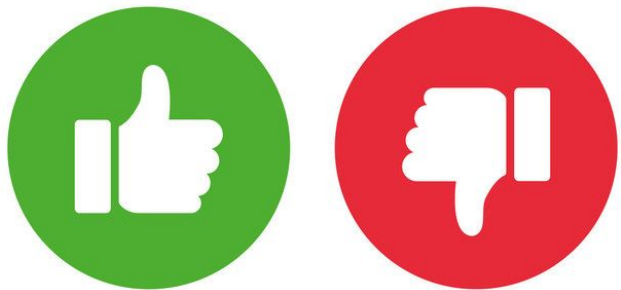
Demo

Transforming a stateless LLM into a stateful agent with basic tool use



Pulse Check

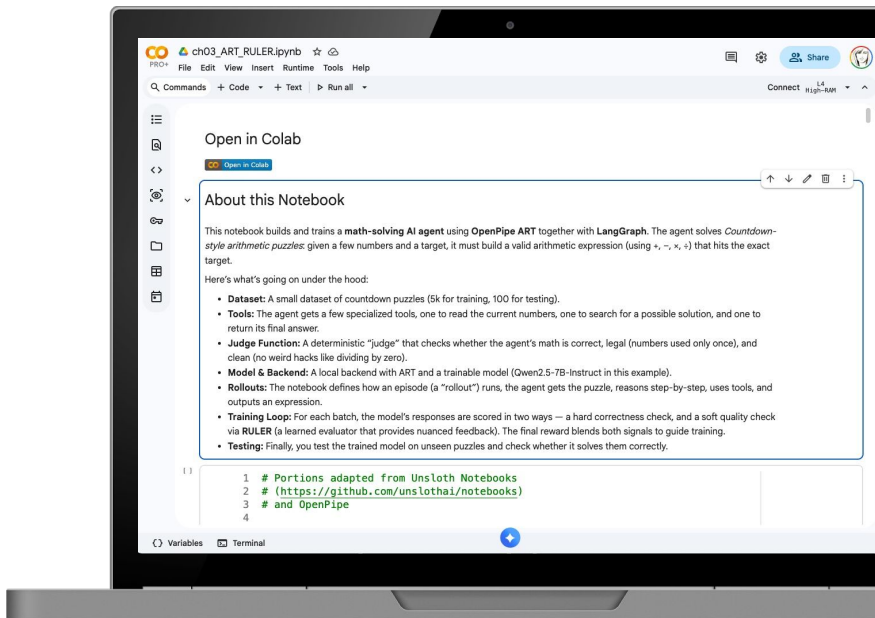
Has everyone set up their Notebook environment?



Hands-on

Decompose a given small task and implement it as a LangGraph workflow with state and one tool

- Instructions are in the Colab Notebook.



Q&A





Break



Session 2

Making the Agent Smarter

Course Schedule

Session 2

Making the agent smarter (60 minutes)

- **Presentation:** Adding intelligence and optimization at test time using structured reasoning patterns such as chain of thought and ReAct
- **Demo:** Implementing chain of thought and ReAct for an agent from scratch
- **Hands-on Exercise:** Compare a simple single agent to a planner; judge setup and observe how structured reasoning and test-time compute affect the quality of the outcome
- **Q&A** (5 min)

Core Components to Make Agents Smarter

- **Perception:** inputs and context
- **Reasoning:** plan and decide next step
- **Action:** tools and code
- **Learning:** short and long term memory

Chain-of-Thought Prompting (CoT)

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

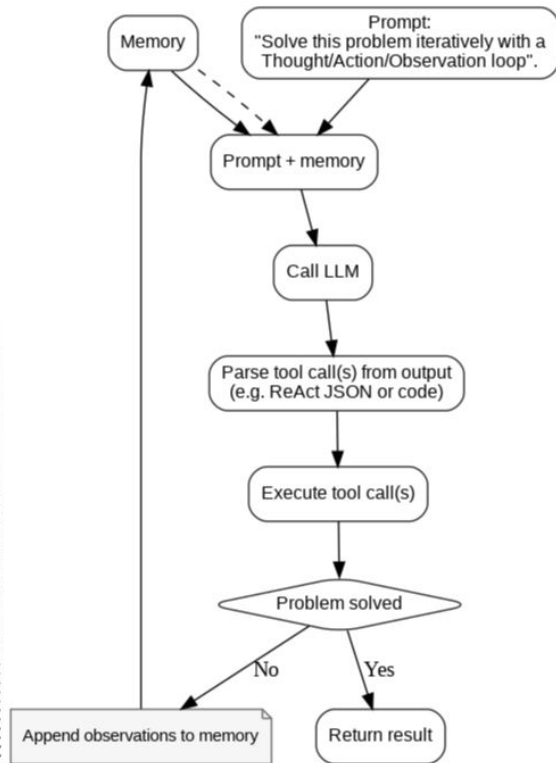
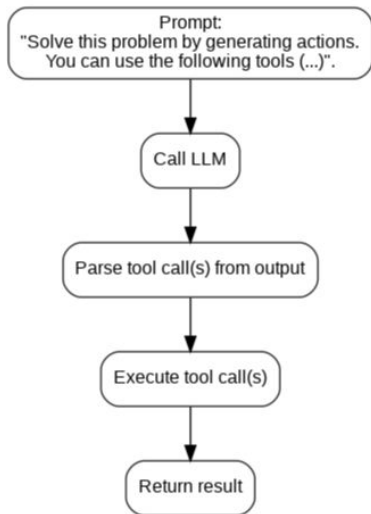
Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

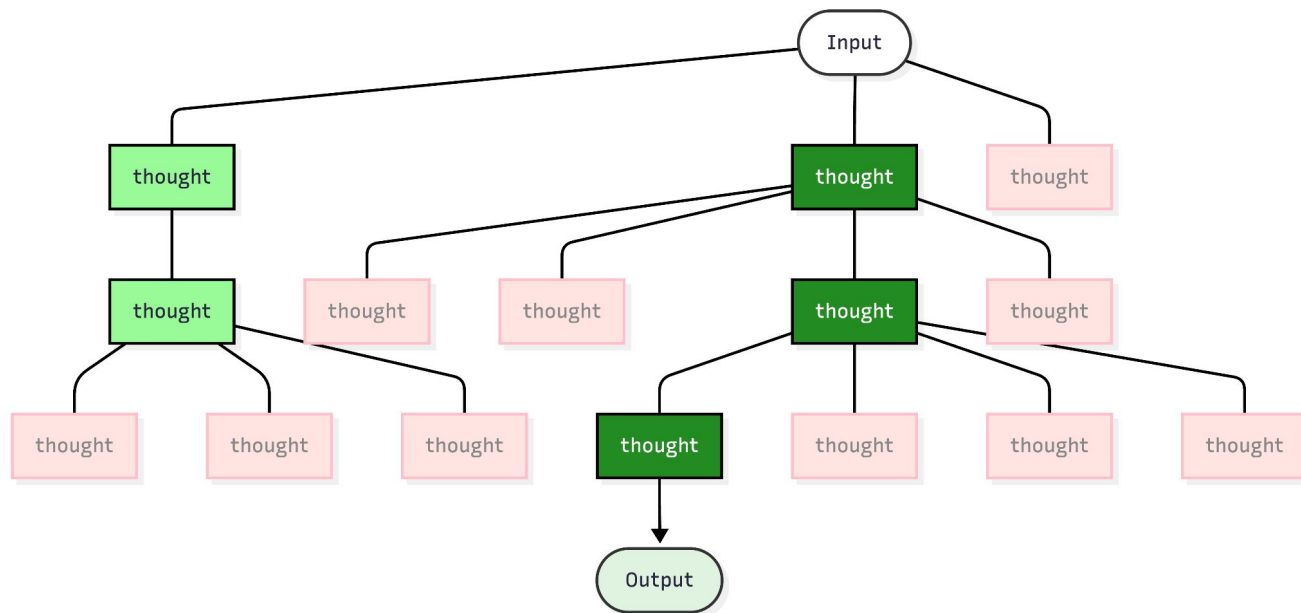
CoT for Agents

Scenario	CoT prompt example
Planning	“Plan step by step to create a one week blog schedule. Explain the reasoning behind each choice.”
Data analysis	“Begin by describing the dataset. Next highlight patterns or trends. Conclude with a clear summary of insights.”
Research task	“List the central questions first. Investigate them one by one. Conclude with a synthesized summary.”
Debugging	“Start by listing possible causes. Test each cause in turn. Eliminate what does not fit and explain why until you reach the fix.”
Decision-making	“List all available options. Consider pros and cons step by step. Choose the option that best fits the criteria and explain why.”

Single Step vs. ReAct Agent

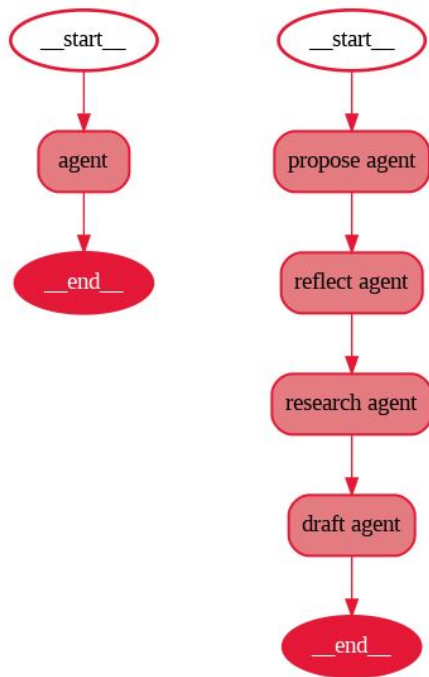


Tree of Thought (ToT) as an Engineering Shortcut



A lightweight ToT setup where a smaller model proposes branches and a stronger model evaluates can sometimes replace the need for a full multi-agent architecture. This simplifies design and saves both money and time by avoiding repeated calls to large reasoning models.

Control-flow Comparing CoT and ToT



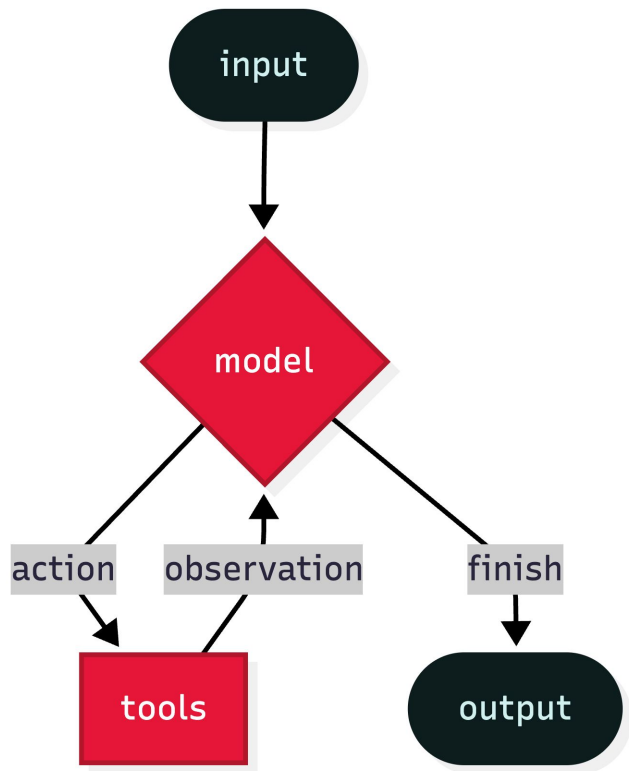
On the left a single agent reasoning with CoT, on the right a multi-agent setup where different roles collaborate to explore, reflect, and refine mimicking ToT.

Example: Single-step vs. Multistep (ReAct)

Task: “Summarize the document and create a bar chart of key statistics”

Step	Single-Step Agent	Multistep (ReAct) Agent
1	Reads the request once	Reads the request and forms an initial Thought about the first action (for example, extract statistics)
2	Plans the entire sequence internally	Action: execute extraction Observation: store extracted data in memory
3	Generates summary and chart in a single pass	Thought: plan next action to generate summary from extracted data
4	Returns outputs without revisiting intermediate results	Action: generate summary Observation: store output in memory
5	No mid-execution adjustments	Thought: plan final action to create bar chart Action: create chart Observation: review output
6	—	If intermediate results are incomplete, repeat the loop until satisfied, then return final outputs

Production-Ready Implementation with LangGraph



`create_agent` constructs a graph based agent runtime with LangGraph. The graph is composed of nodes (steps) and edges (connections) that define how information flows through the system. As the agent runs, it transitions between nodes such as the model node, which invokes the model, the tools node, which executes external tools, and optional middleware that handles intermediate logic.

Out of the Box ReAct Engine with LangGraph

LangGraph's `create_agent` already gives you a production grade ReAct engine out of the box.

This is not prompt choreography and it is not a recursive LLM wrapper pretending to be an architecture. It is a compiled state machine with a native tool calling loop, typed state management, structured output validation, middleware interception points, and built in checkpointing.



Compiled ReAct Control Loop

With `create_agent` you get a proper state machine:

Model → Tool Calls → Tool Execution → Observation → Model → ... → Stop

This is a structured ReAct loop with:

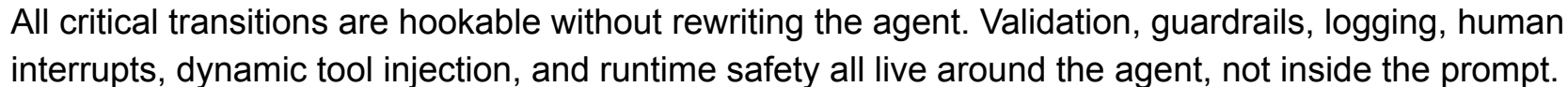
- No fragile string parsing
- Native tool call handling
- Explicit stop condition
- Deterministic control flow

Built-in Structured Output Handling

You get out of the box:

- Pydantic schemas
- ToolStrategy
- ProviderStrategy
- Auto strategy detection
- Validation + retry logic

That means structured output is not an afterthought. It is part of the execution loop.



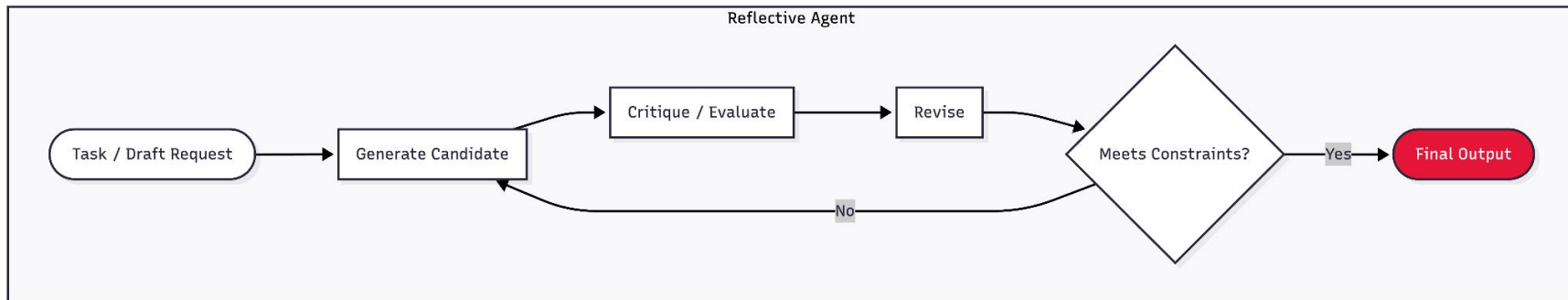
Checkpointing and Cross-Thread Memory

You get out of the box:

- Checkpointers for conversation state
- Stores for multi-session persistence

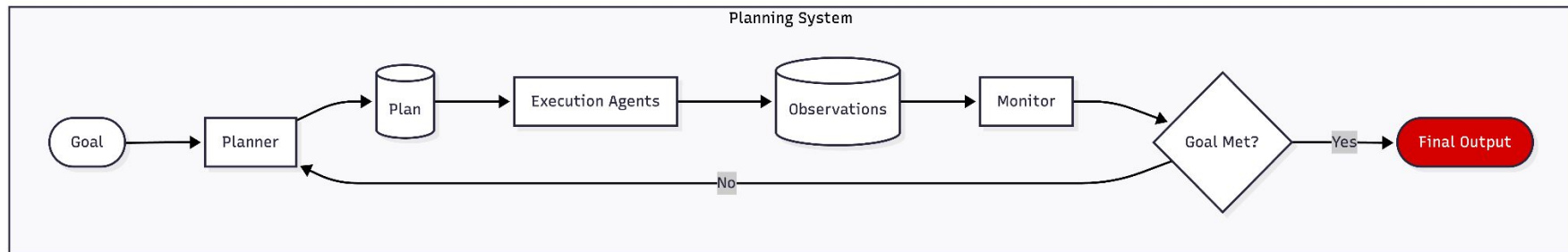
This means production memory is native to the agent, not bolted on.

Reflective Agent Loop



Iterative self-improvement: A single agent evaluates and refines its own output. Reduces error propagation in precision tasks, but does not solve coordination failures in distributed systems.

Planning System



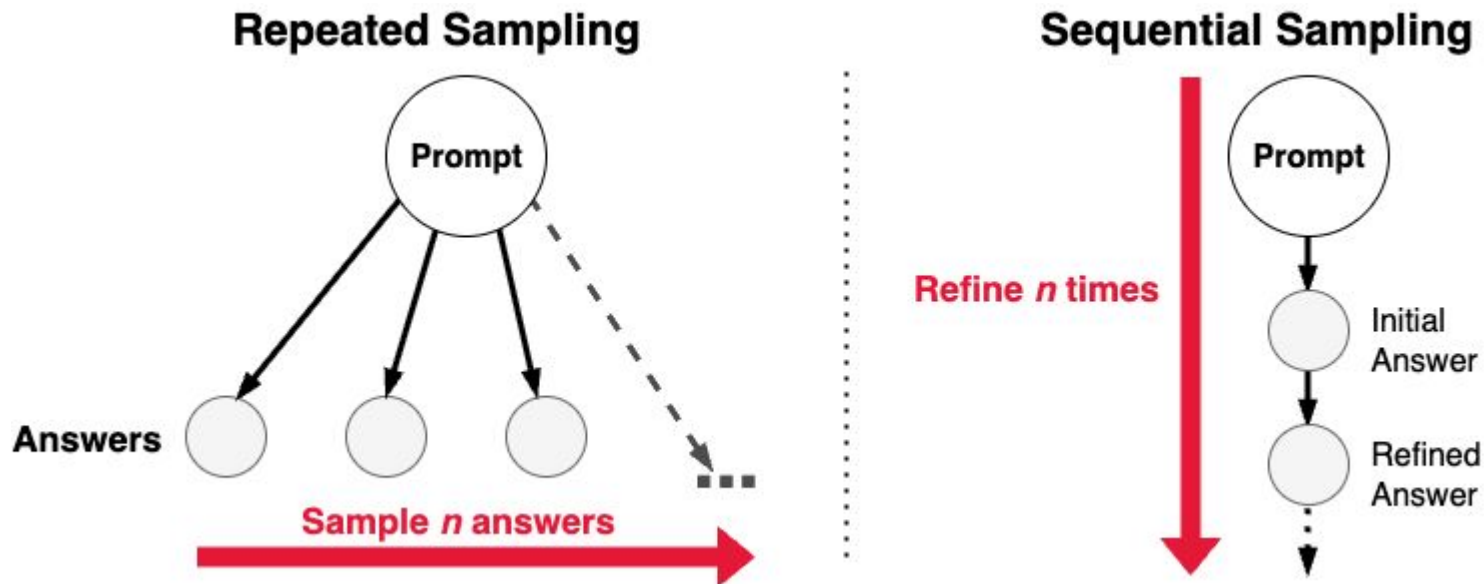
Goal-driven coordination: A planner creates multi-step strategy, executors act, monitor evaluates progress. Powerful for long-horizon tasks, but planners can collapse under excessive reasoning load.



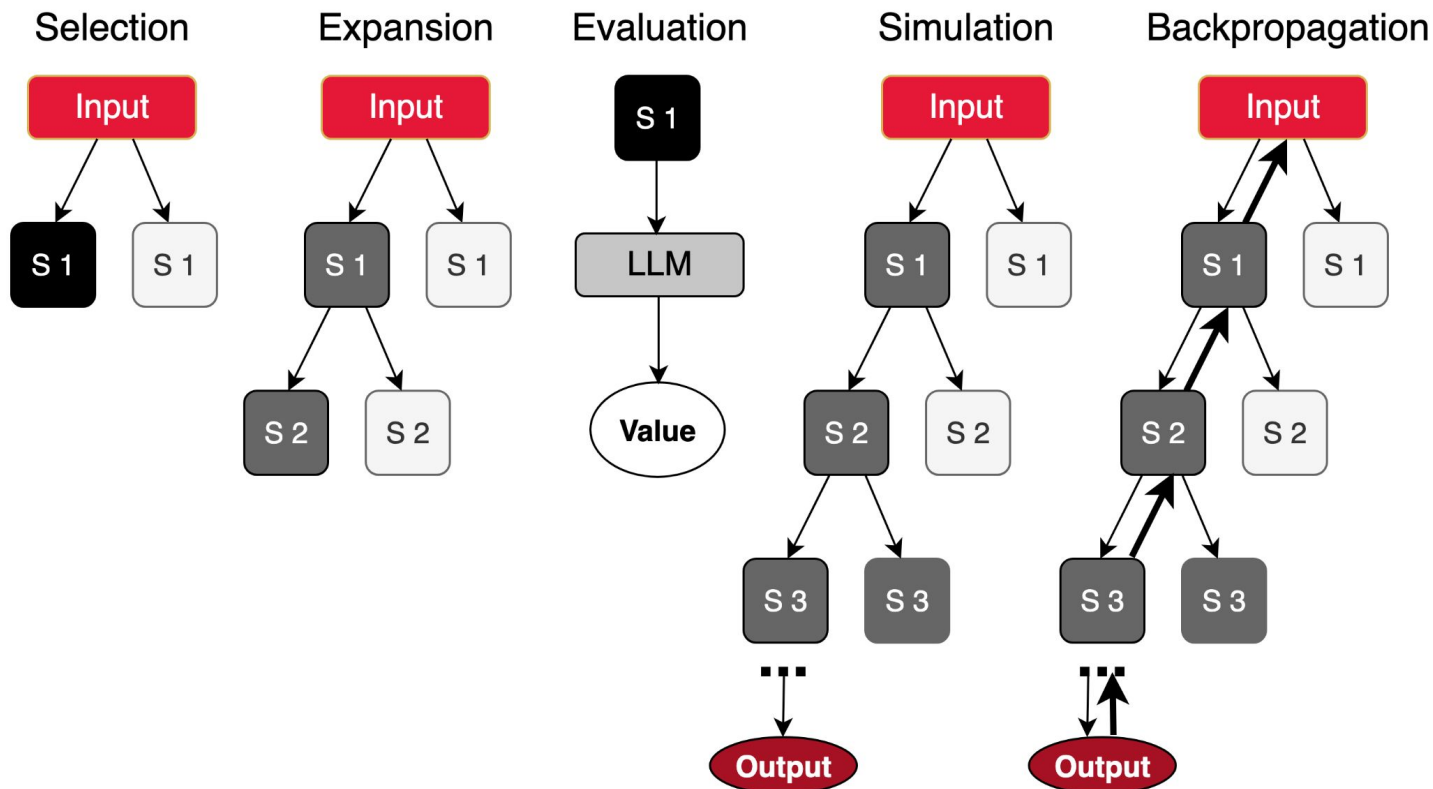
Test-time Compute

Test-time: Sampling Strategies

Repeated sampling versus sequential refinement



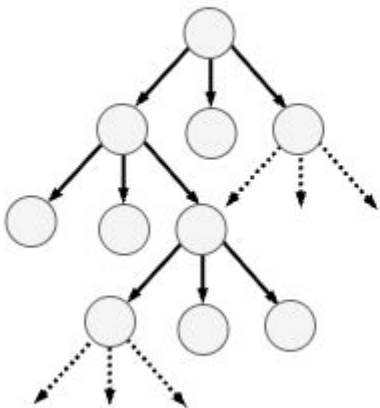
Test-time: Tree-based Search



Test-time: Sampling Strategies

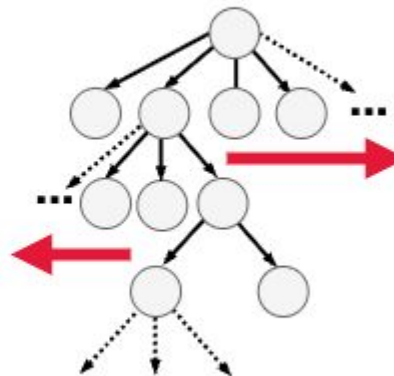
Comparison of MCTS with Adaptive MCTS

Standard MCTS



Fixed branching factor (3)

Adaptive Branching MCTS



Branch adaptively, when appropriate

Test-time Compute with RULER

Relative Universal LLM-Elicited Rewards (RULER) is a new general-purpose reward function that, when used with GRPO, can reliably improve models with no labeled data, no hand-crafted reward functions, and no human feedback. This dramatically simplifies the process of adapting RL to a new task.

RULER

Judging Cat Poems with RULER

Rank 1: Score 0.900

Response: Upon the roof, their eyes like stars,
They watch the sky from silver jars.
Each ...

Rank 2: Score 0.600

Response: Cats sit and look at stars at night,
They think the stars are shiny and bright....

Rank 3: Score 0.050

Response: Dogs are great companions under the moon. They bark at stars and wag their tails...

JSON Evaluation Choice for Cat Poems

```
{
  'scores': [
    {
      'trajectory_id': '1',
      'explanation': 'Poetic, cat-focused, night-sky theme fulfilled with vivid imagery; brief and compliant.',
      'score': 0.9
    },
    {
      'trajectory_id': '2',
      'explanation': 'Meets topic but very simplistic and minimally evocative; partial fulfillment.',
      'score': 0.6
    },
    {
      'trajectory_id': '3',
      'explanation': 'Ignores cat theme, focuses on dogs; fails the main instruction.',
      'score': 0.05
    }
  ]
}
```



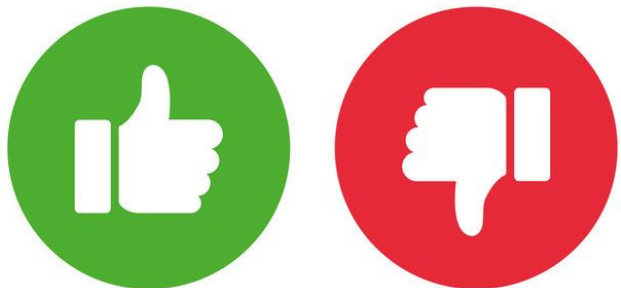
Key Takeaways Presentation Session 2

Making the agent smarter

- Intelligence is structured search
- Test-time compute trades latency for reliability
- Validation lowers propagation error

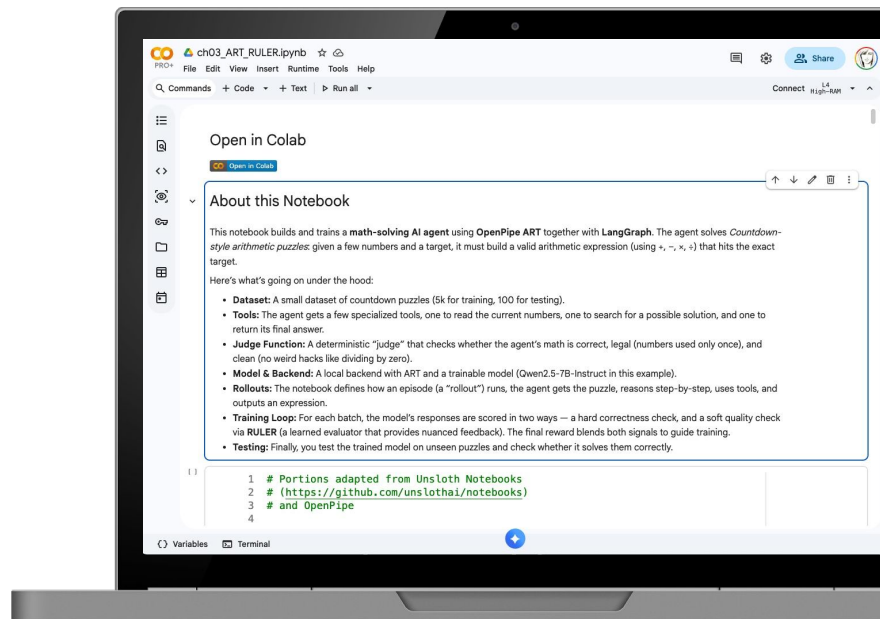
Pulse Check

Do you have basic understanding how to make your agent smarter?



Demo

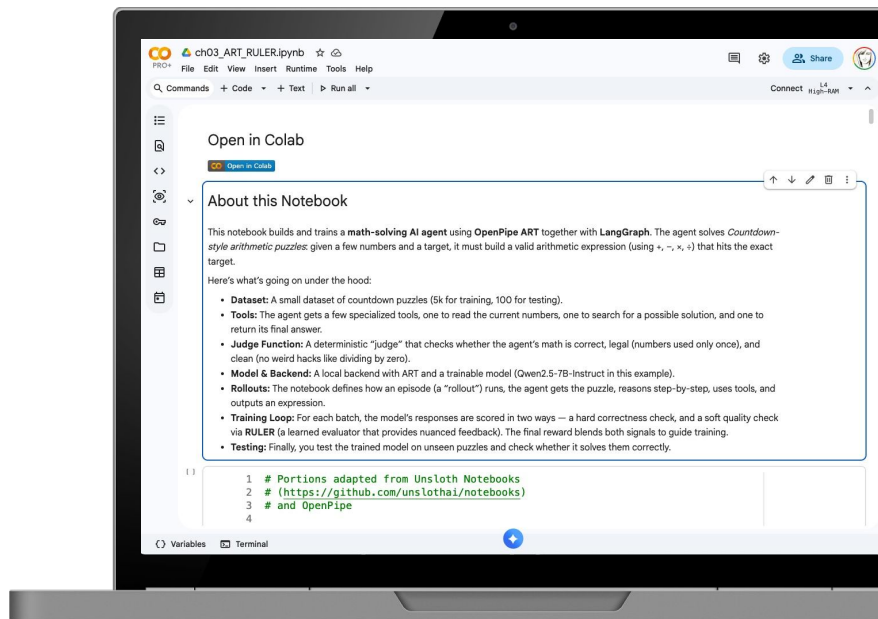
Implementing chain of thought and ReAct for an agent from scratch



Hands-on

Compare a simple single agent to a planner; judge setup and observe how structured reasoning and test-time compute affect the quality of the outcome

- Instructions are in the Colab Notebook.



Q&A





Break



Session 3

Adding Human Oversight to Your System

Course Schedule

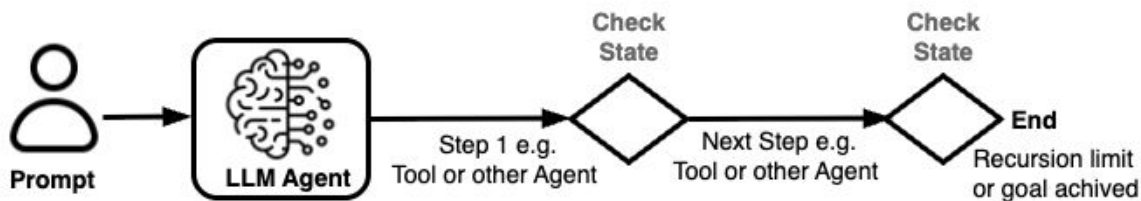
Session 3

Adding human oversight to your system (60 minutes)

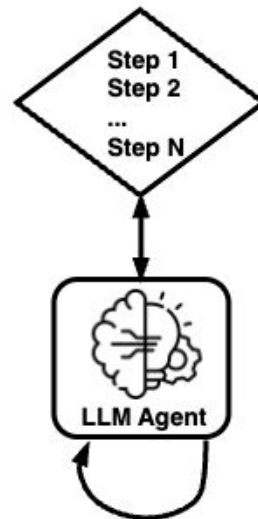
- **Presentation:** How much autonomy agents have in practice; implementing safeguards with human-in-the-loop patterns
- **Demo:** Adding human-in-the-loop checkpoints in LangGraph to approve or reject actions, edit graph state, and request additional human input before sensitive steps
- **Hands-on Exercise:** Insert a human-in-the-loop gate into your own agent workflow
- Q&A (5 mins)

Orchestrated Autonomy

Orchestrated Autonomy



Fully Autonomous



In orchestrated autonomy, decision points are predefined in the workflow. In full autonomy, the LLM could generate new tools, actions, or decision paths to reach its goal.

Orchestrated vs. Unbounded Autonomy

You can build clear state transitions, deterministic control flow, validation layers, tool boundaries, and execution tracing into any serious agent system (“fully” autonomous or not). The difference is this:

- Orchestrated autonomy defines explicit escalation boundaries where authority shifts beyond the agent.
- Introduces intentional intervention points for high-impact decisions, irreversible actions, or ambiguity.
- Agentic system can't continue or decide on its own.

Full autonomy optimizes for continuity.

Orchestrated autonomy optimizes for accountability.

Autonomy: Security Risks Examples

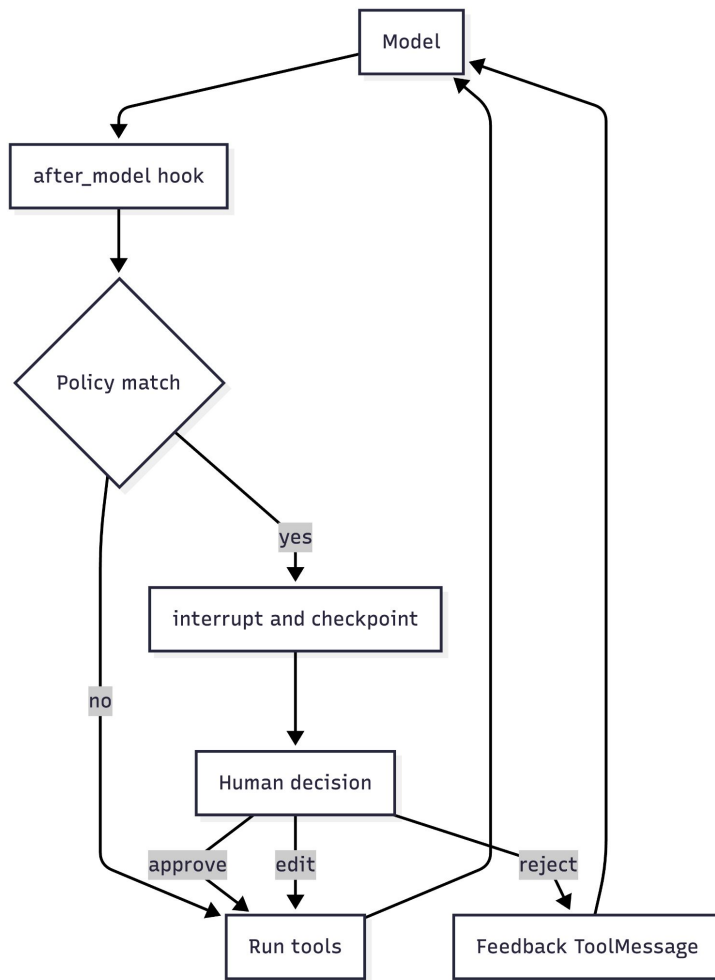
- Agents often act with elevated privileges on behalf of users.
- Risk of trust boundary violations when agents delegate across APIs or tools.

Problem examples:

- An agent allowed to execute unchecked DB queries is manipulated via injected instructions to run high-privilege queries.
- Agent has higher privileges than the user, and is tricked into executing unauthorized actions.

HITL Lifecycle

In orchestrated autonomy, decision points are predefined in the workflow. In full autonomy, the LLM could generate new tools, actions, or decision paths to reach its goal.



Interrupt Decisions Overview

A human can approve, edit, or reject each paused action. Decisions are applied in order.



approve

Run the tool call as proposed.

Example

Send the email draft as written



edit

Run the tool call with conservative edits to args.

Example

Change recipient before sending



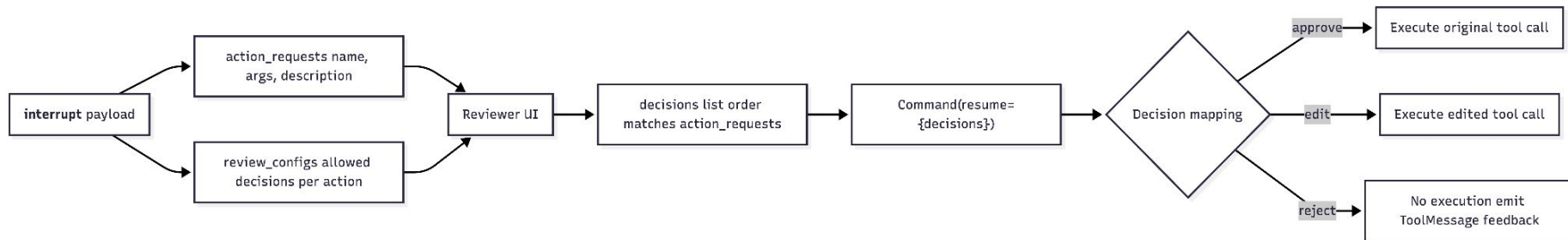
reject

Reject with feedback added to the conversation.

Example

Block a dangerous SQL statement

HITL Interrupts with Resume



When an interrupt is raised and the graph state is checkpointed. Execution resumes only after an explicit `Command(resume=...)` decision is provided, ensuring safe continuation.

What Exactly is Interrupt?

- Interrupt includes `action_requests` (tool name, args, description)
- `review_configs` (allowed decisions per action)
- Multiple tool calls require multiple decisions in the same order.

Resume Semantics (Approve/Edit/Reject)

- Approve executes the original tool call
- Edit executes the tool call with modified args
- Reject does not execute; instead it injects a ToolMessage containing the rejection explanation,

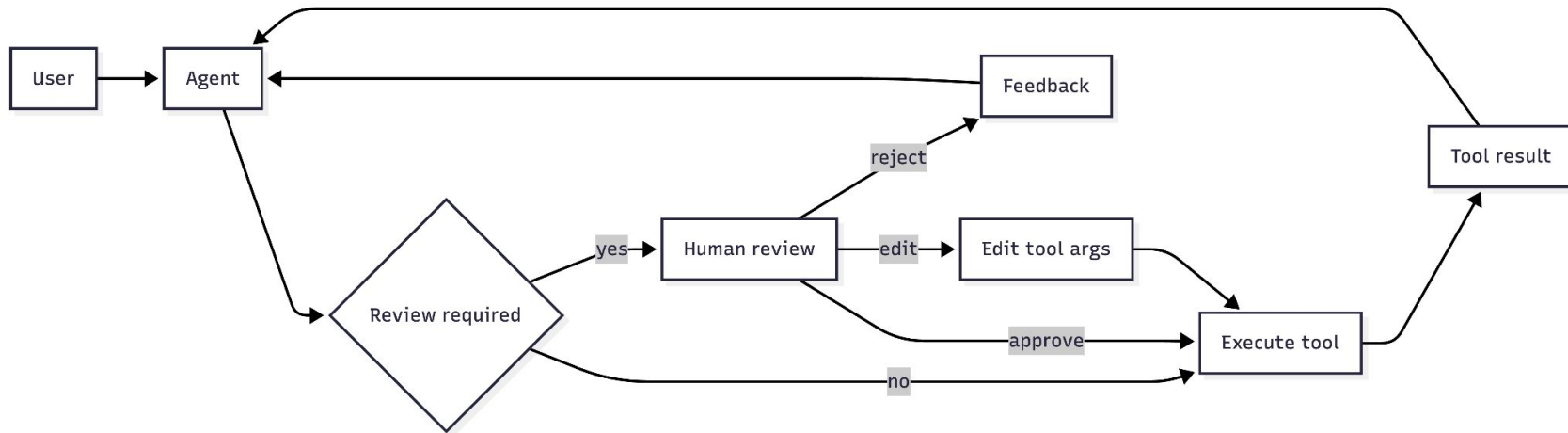
The agent can re-plan without performing the risky action

Interrupt Decisions in Detail

There are typically three different actions that you can do with a human-in-the-loop workflow:

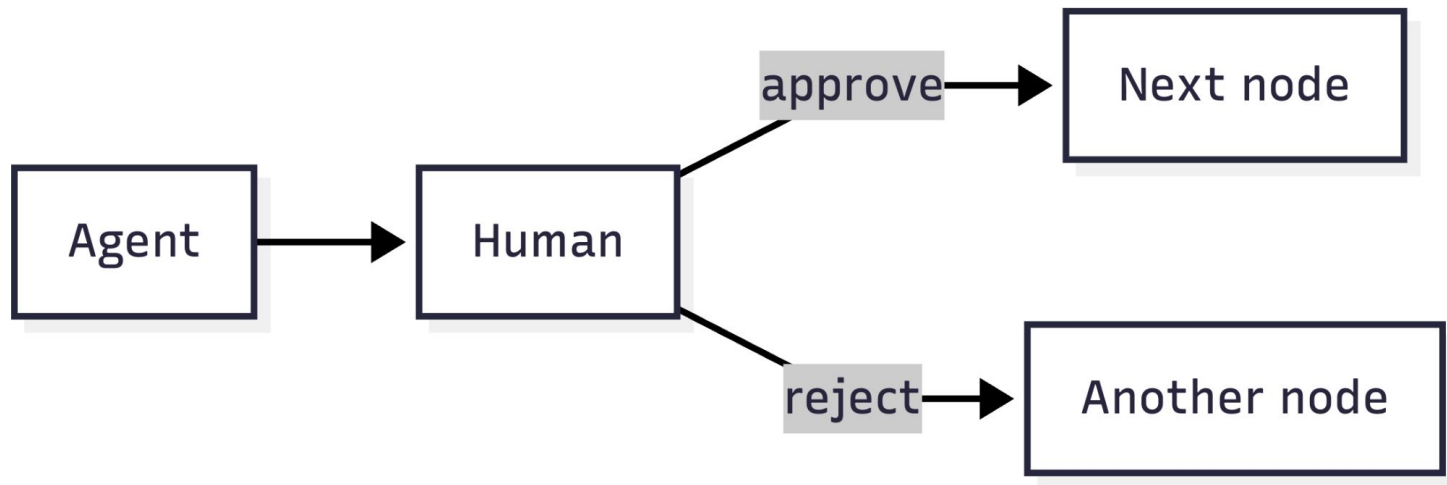
- **Approve or Reject:** Pause the graph before a critical step, such as an API call, to review and approve the action. If the action is rejected, you can prevent the graph from executing the step, and potentially take an alternative action. This pattern often involve routing the graph based on the human's input.
- **Edit Graph State:** Pause the graph to review and edit the graph state. This is useful for correcting mistakes or updating the state with additional information. This pattern often involves updating the state with the human's input.
- **Get Input:** Explicitly request human input at a particular step in the graph. This is useful for collecting additional information or context to inform the agent's decision-making process or for supporting multi-turn conversations.

Behavior View of HITL Control



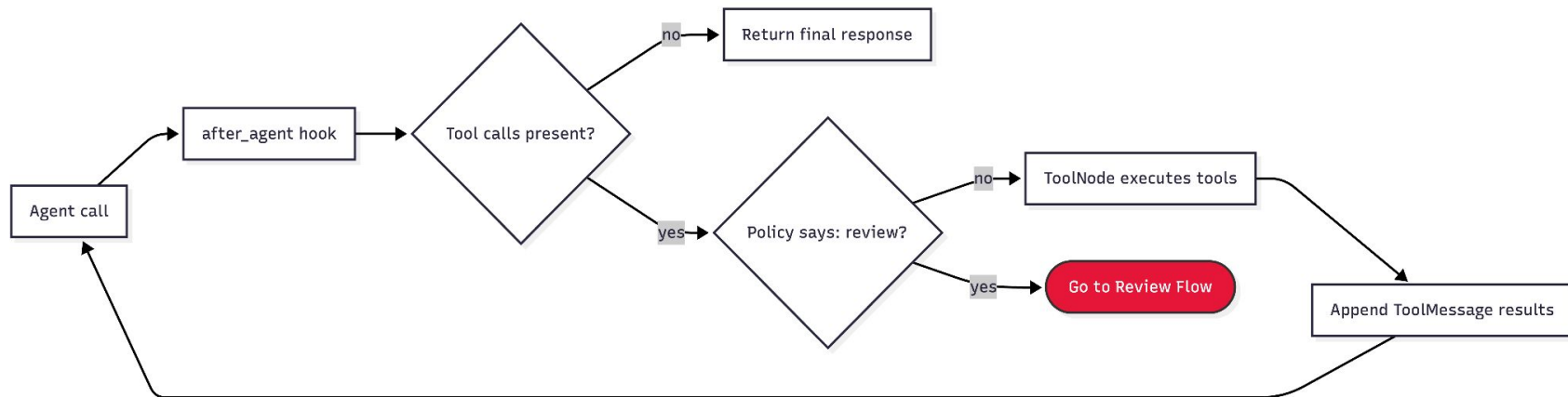
The Agent proposes an action, a policy determines whether review is required, and execution either proceeds automatically or pauses for human oversight. Approved or edited actions execute, while rejected actions inject feedback back into the reasoning loop.

Authority Boundary Introduced by HITL



The human reviewer acts as a control node that determines the next state transition. The system does not simply continue execution. Authority shifts from the agent to an external decision point before proceeding.

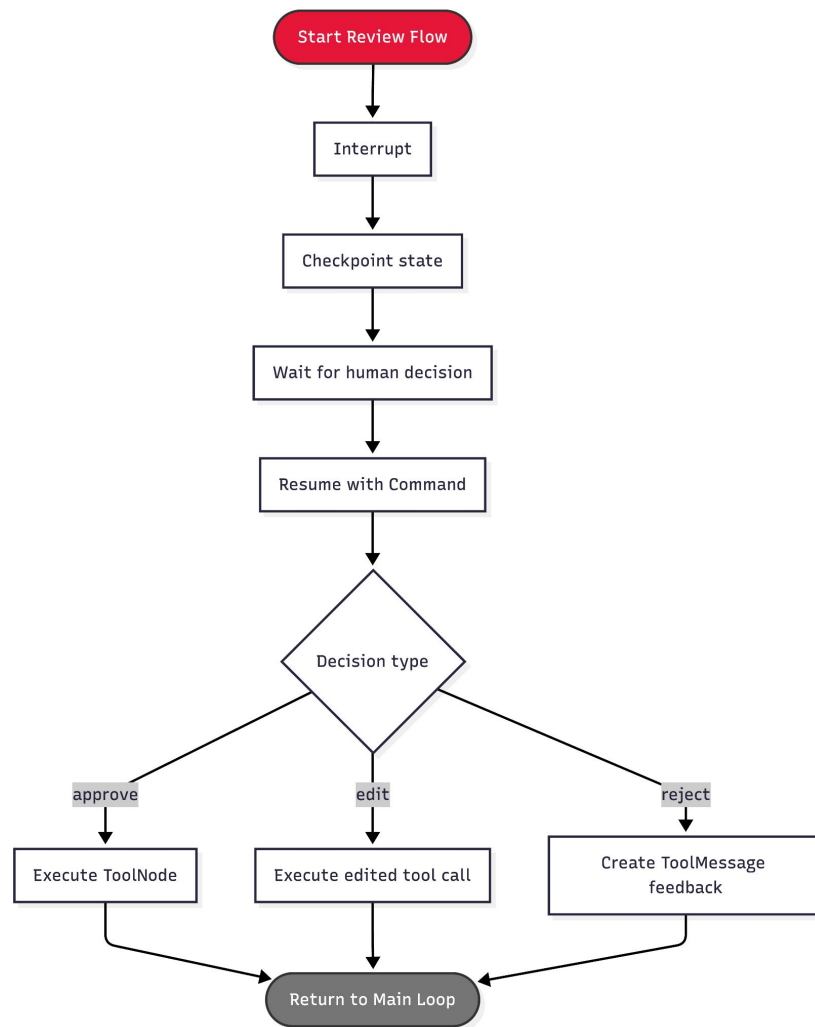
HTIL Detection and Automated Execution



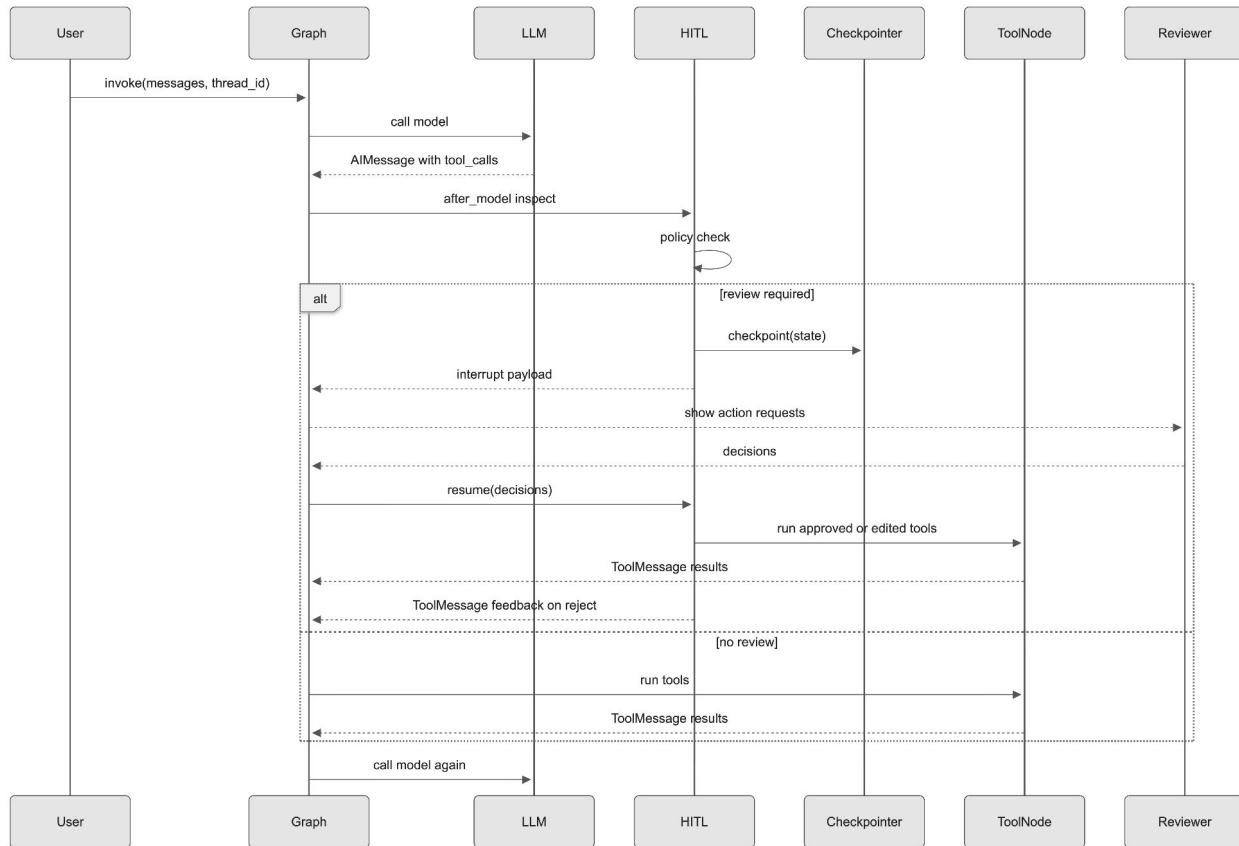
This graph focuses on the primary agent loop, The connector node shows where the flow hands off to the review process (next slide).

HITL Process

This graph covers the state management and human intervention. It ends by showing how the decision feeds back into the main cycle.



HITL Execution Lifecycle



HITL Execution Lifecycle in Detail

- Agent runs and proposes tool calls
- `after_model` middleware inspects tool calls
- If policy matches, build `HITLRequest` and call `interrupt()`
- `Interrupt` returns interrupt payload to the caller
- Human returns decisions (approve/edit/reject) in the same order
- Middleware applies decisions: execute, execute edited, or synthesize rejection `ToolMessage`
- Graph resumes and the agent continues the loop

Orchestrated Autonomy

You can build strong validation and deterministic control flow with or without humans. The distinctive value of orchestrated autonomy is explicit escalation boundaries where authority shifts beyond the agent for high-impact actions.





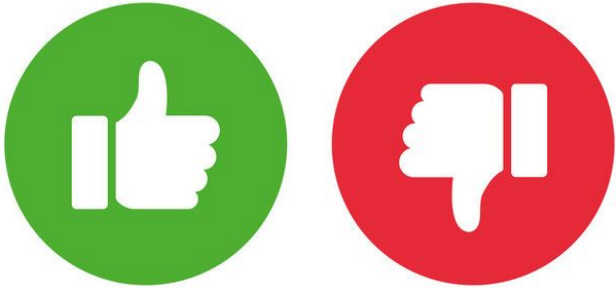
Key Takeaways Presentation Session 3

Adding human oversight to your system

- Autonomy without escalation is risk
- Authority boundaries must be explicit
- Interrupt $\neq$ failure; it is governance

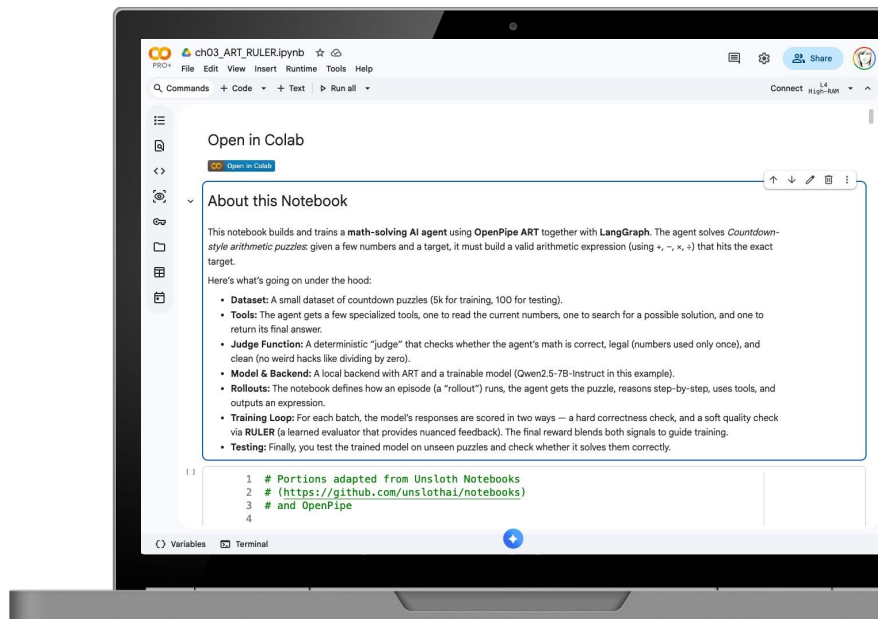
Pulse Check

Do you see the benefits of HITL?



Demo

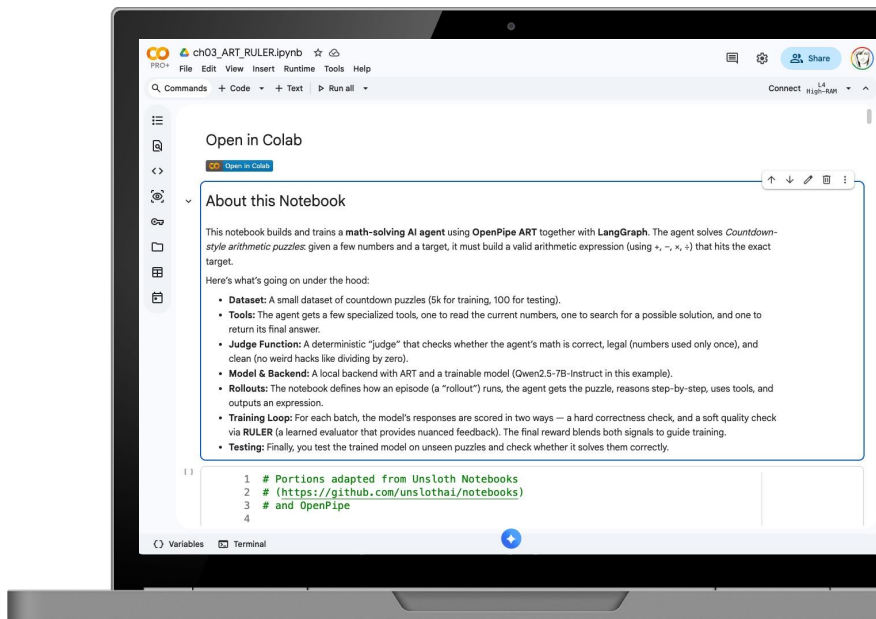
Adding human-in-the-loop checkpoints in LangGraph to approve or reject actions, edit graph state, and request additional human input before sensitive steps



Hands-on

Insert a human-in-the-loop gate into your own agent workflow

- Instructions are in the Colab Notebook.



Q&A





Break

Session 4

From Single Agents to Multi-agent Systems

Course Schedule

Session 4

From single agents to multi-agent systems (60 minutes)

- **Presentation:** Scaling from a single, tool-using agent to a multi-agent system using collaboration patterns; when to apply each
- **Demo:** Building a supervisor-based multi-agent system in LangGraph where a coordinator agent delegates tasks to specialist agents
- **Hands-on Exercise:** Turn your existing agent into a two-agent workflow and test how responsibilities and handoffs change system behavior
- **Q&A** (5 mins)

Why Collaboration Patterns Matter

MAS = not just multiple agents, but structured coordination.

Collaboration patterns define:

- How work is divided
- How decisions are reconciled
- How failures are contained

Different real-world problems → different collaboration patterns.

The System-Level Insight

Patterns are not stylistic choices. They define information flow and scaling behavior. Adding agents does not behave like adding parameters. More agents → more coordination tax:

- Communication overhead
- Latency
- Context blowup
- Error amplification

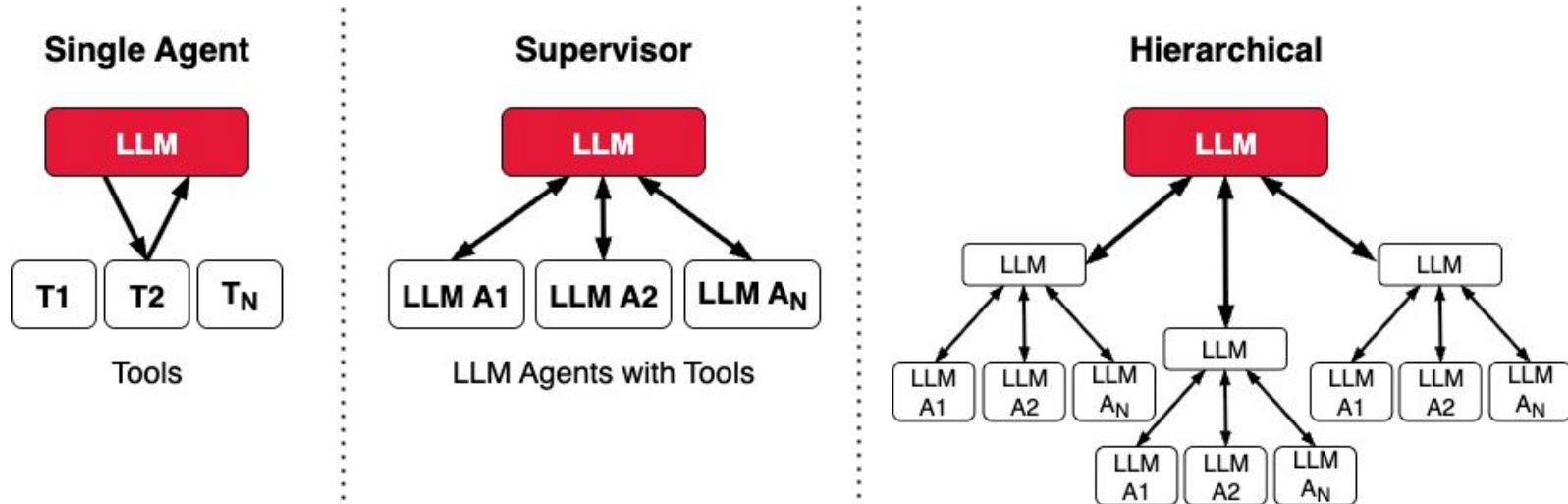




The Four Core Patterns

- Hierarchical delegation
- Peer-to-peer architecture
- Blackboard architectures
- Swarm intelligence

Hierarchical Delegation



One supervisor agent delegates tasks to specialized sub-agents. Supervisor decides sequencing, error handling, and integration of outputs.

Example Hierarchical (Supervisor) Agent Flow

Supervisor agent:

- Interprets user objective
- Decomposes task

Delegates to sub-agents:

- Retrieval agent
- Planning agent
- Execution agent
- Validation agent
- Compliance / safety agent

Hierarchical Delegation: Pros & Cons

The strength of this pattern is control. The weakness is that every decision becomes a bottleneck.

Pros:

- Clear accountability
- Simplified coordination
- Fits regulated environments

Cons:

- Bottleneck at supervisor
- Risk of single point of failure
- Less adaptive to unexpected events

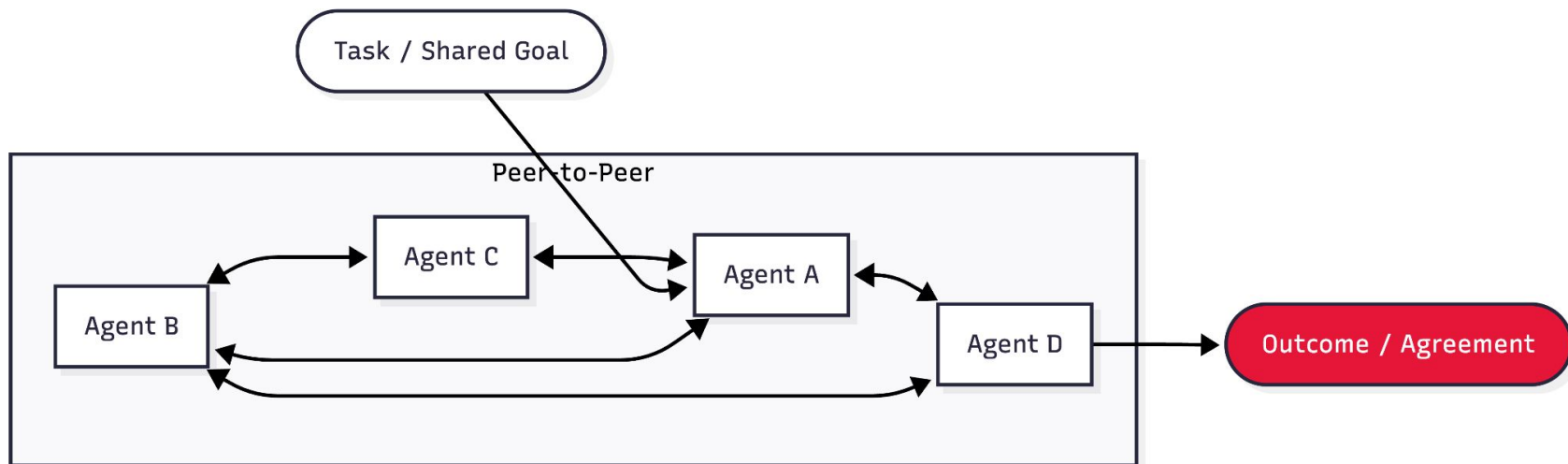
Supervisor Agent Flow – Where It Works Well

Some examples are:

- Compliance workflows
- Enterprise approval pipelines
- Legal document review
- Medical decision support
- Insurance claims processing
- Structures data analysis

This pattern usually fails when the task becomes exploratory or creative.

Peer-to-peer Collaboration



Decentralized negotiation: Agents exchange information directly without a central controller.
Adaptive and resilient, but requires explicit convergence or validation mechanisms to prevent drift.

Example Peer-to-peer Agent Flow

- Agent A optimizes cost
- Agent B optimizes speed
- Agent C optimizes robustness
- Agents negotiate trade-offs

Peer-to-peer Collaboration: Pros & Cons

Agents act as equals.

Each agent has local objectives, they negotiate, trade, or coordinate.

Pros:

- Resilient (no single bottleneck)
- Adaptive to changing markets

Cons:

- Complex coordination overhead
- Harder to regulate & audit
- Risk of collusion between agents

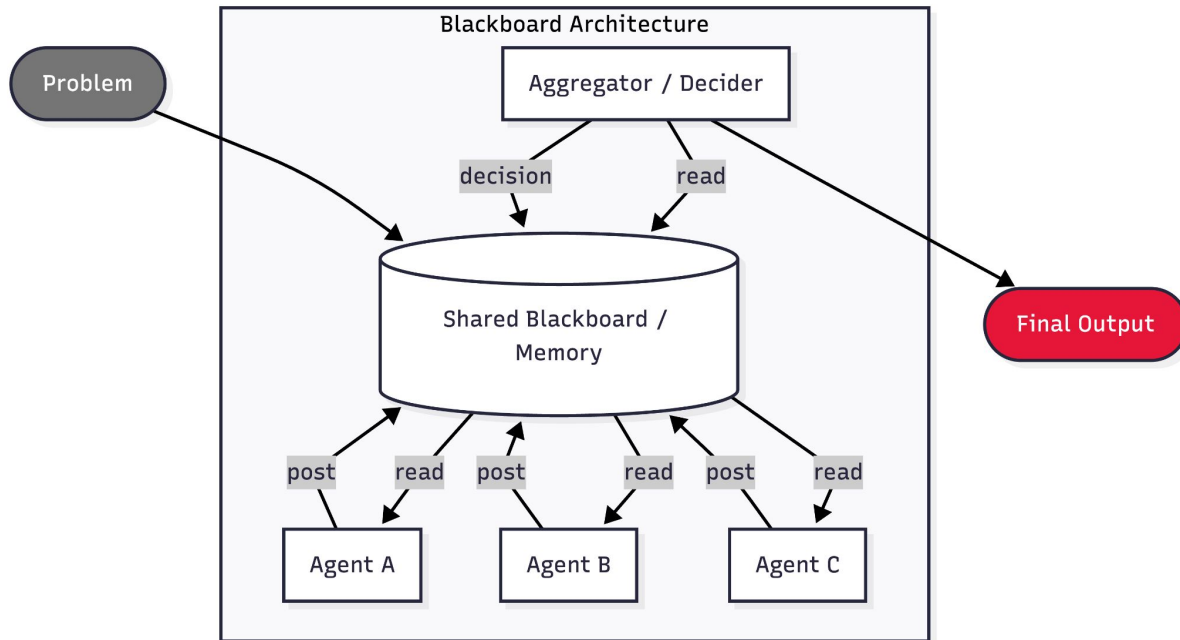
Peer-to-peer Collaboration – Where It Works Well

Some examples are:

- System design exploration
- Product ideation
- Architecture search
- Open-ended research
- Web navigation

**Without aggregation or validation, peer systems fragment.
You need consolidation checkpoints.**

Blackboard Architecture



Shared workspace reasoning: Agents externalize intermediate thoughts into shared memory. Enables accumulation and critique, but requires synchronization discipline and termination logic.

Example Blackboard Architecture

Shared memory object:

- Research agent posts findings
- Drafting agent writes proposal
- Critique agent annotates issues
- Validator updates confidence score

Blackboard Architecture: Pros & Cons

Blackboard architecture is inspired by human experts collaborating and solving difficult problems by sharing information and contributing their expertise.

Pros:

- Transparent, auditable
- Agents can build on each other's results

Cons:

- Central point of failure
- Risk of overload / inconsistency
- Synchronization challenges

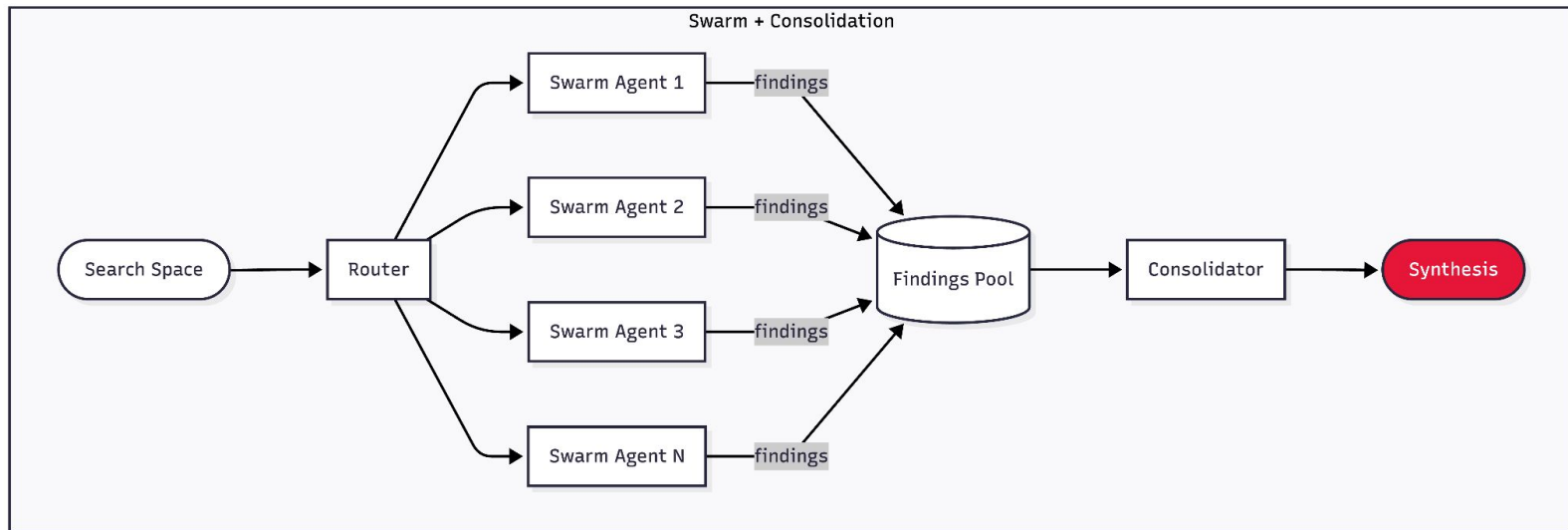
Blackboard Architecture – Where It Works Well

Some examples are:

- Creative writing
- Strategic planning
- Research synthesis
- Policy drafting
- Complex debugging

**Blackboard systems mirror how creative teams operate.
But without termination logic, they burn tokens endlessly.**

Swarm Architecture



Parallel exploration: Many lightweight agents search simultaneously. Redundancy improves coverage and signal validation. Requires strict exit conditions and a downstream aggregator to prevent token explosion.

Example Swarm Architecture

MAS Example:

- 20 lightweight agents crawl web sources
- Each surfaces candidate answers
- Redundancy used for signal validation
- Aggregator consolidates

Swarm Architecture: Pros & Cons

**Many simple agents, local interactions → emergent global behavior.
Inspired by biological swarms (ants, bees, birds).**

Pros:

- Scalable, robust to failures
- Good for simulation & forecasting

Cons:

- Hard to control or predict
- Difficult to audit or regulate
- Emergent outcomes may diverge from intent

Swarm Architecture – Where It Works Well

Some examples are:

- Web research
- Market scanning
- Threat detection
- Trend discovery
- Large search spaces

**Swarms are excellent for coverage, not convergence.
Always pair with a consolidation phase.**

Comparative Overview

Pattern	Best For	Strength	Failure Mode
Reflective	Precision tasks	Error reduction	Slow, limited scope
Hierarchical	Compliance, pipelines	Control	Bottleneck
Peer-to-peer	Exploration	Flexibility	Drift
Blackboard	Creative / synthesis	Accumulation	Memory overload
Swarm	Coverage tasks	Scalability	Token explosion

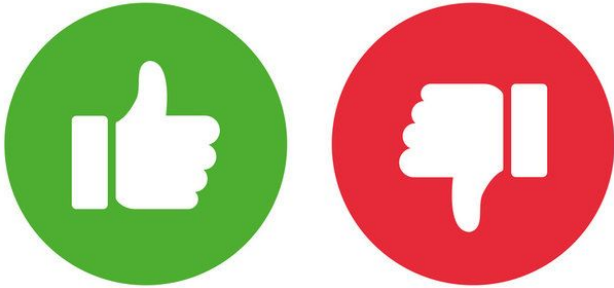
Key Takeaways Presentation Session 4

From single agents to multi-agent systems

- More agents $\neq$ more capability
- Coordination tax is real
- Pattern defines scaling behavior

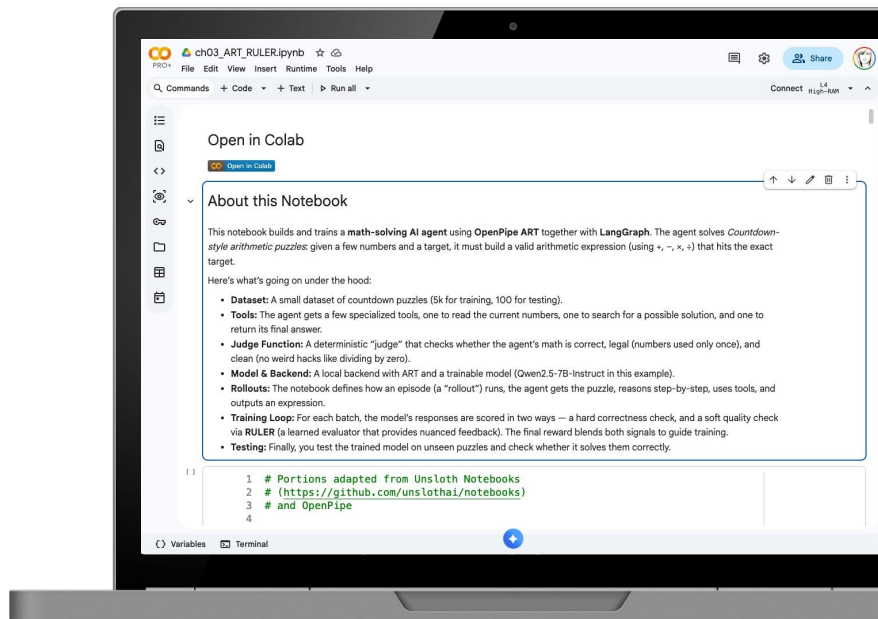
Pulse Check

Are the benefits of collaboration patterns understandable?



Demo

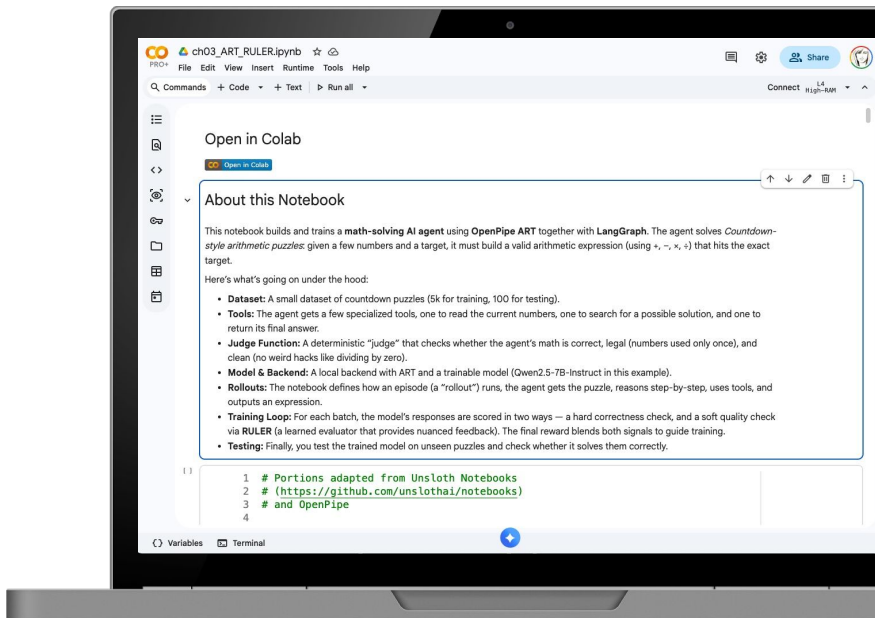
Building a supervisor-based multi-agent system in LangGraph where a coordinator agent delegates tasks to specialist agents



Hands-on

Turn your existing agent into a two-agent workflow and test how responsibilities and handoffs change system behavior

- Instructions are in the Colab Notebook.



Q&A





Break

Session 5

Communication Patterns in Multi-agent Systems

Course Schedule

Session 5

Communication patterns in multi-agent systems (60 minutes)

- **Presentation:** How multi-agent systems coordinate through message passing, shared memory access patterns, and event-driven communication; when to choose each method; how communication shapes collaboration patterns, latency, handoff reliability, and system throughput
- **Demo:** How different multi-agent collaboration patterns work together with communication patterns; which ones to combine for which use case
- **Hands-on Exercise:** Exchange a communication pattern in a predefined MAS and observe how it affects agent coordination and communication
- **Final Q&A** (5 mins)



Why Communication Matters

Collaboration patterns define who works with whom.
Communication protocols define how information flows.

Poor communication design causes:

- Latency spikes
- State inconsistencies
- Error amplification
- Silent failure

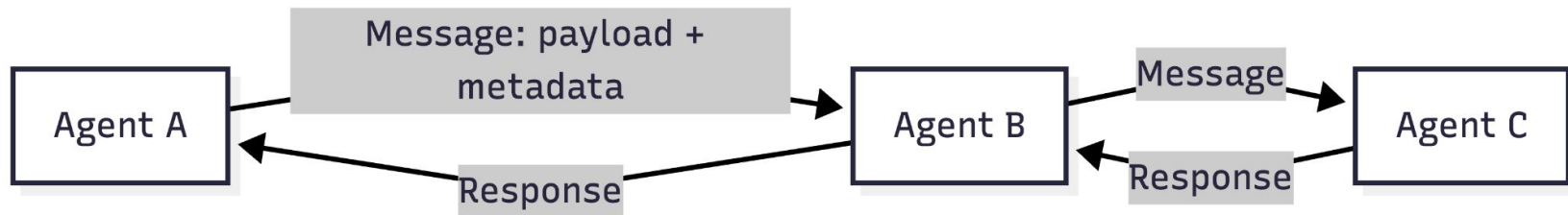
This is where many production MAS collapse.



The Three Core Patterns

- Message Passing
- Shared Memory
- Event-Driven Coordination

Message Passing Systems



Explicit point-to-point communication: Each message contains structured intent and metadata. Strong modularity and traceability, but introduces routing overhead and coordination cost at scale.



Example Message Passing System

Microservices API Calls in a distributed web application:

- Service A calls Service B via API
- Service B returns response
- Each request contains structured payload + metadata

Message Passing System: Pros & Cons

**No shared global memory.
Everything is explicit.**

Pros:

- Clear contracts
- Inspectable logs
- Modular components
- Strong isolation

Cons:

- Message storms
- Latency accumulation
- Duplicate or lost messages
- Complex dependency graphs

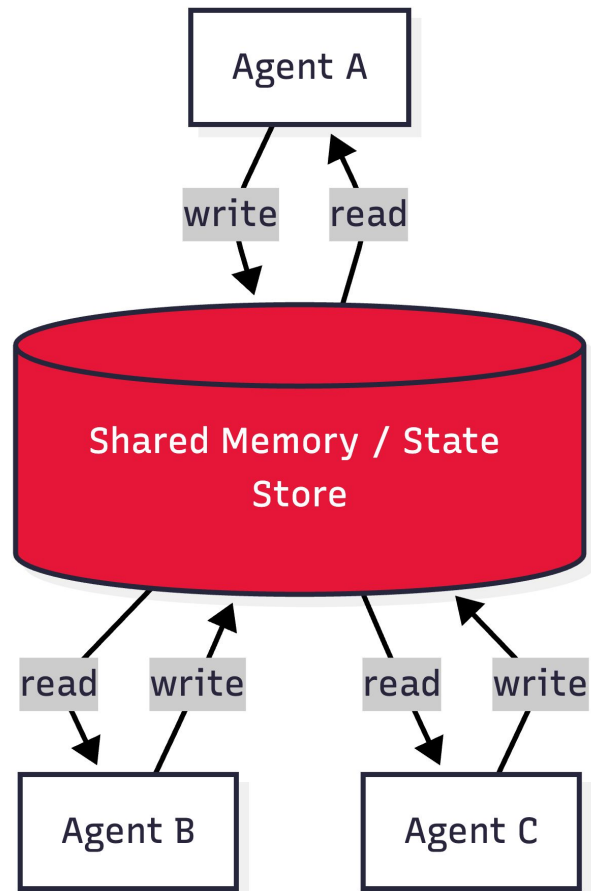
Message Passing Systems – Where It Works Well

Some examples are:

- Tool invocation
- API orchestration
- Workflow pipelines
- Task delegation with explicit contracts

Shared Memory Approaches

Centralized state model: Agents read and write to a shared memory object. Transparent and easy to audit, but vulnerable to synchronization issues and central bottlenecks.



Example Shared Memory Approach

Collaborative Document Editing, where multiple agents editing a shared document:

- One writes section A
- One edits formatting
- One leaves comments
- Everyone sees current state

Shared Memory Approach: Pros & Cons

Single shared source of truth.

Pros:

- Unified state visibility
- Simplified coordination
- Natural logging layer

Cons:

- Race conditions
- State corruption
- Memory bloat
- Single point of failure

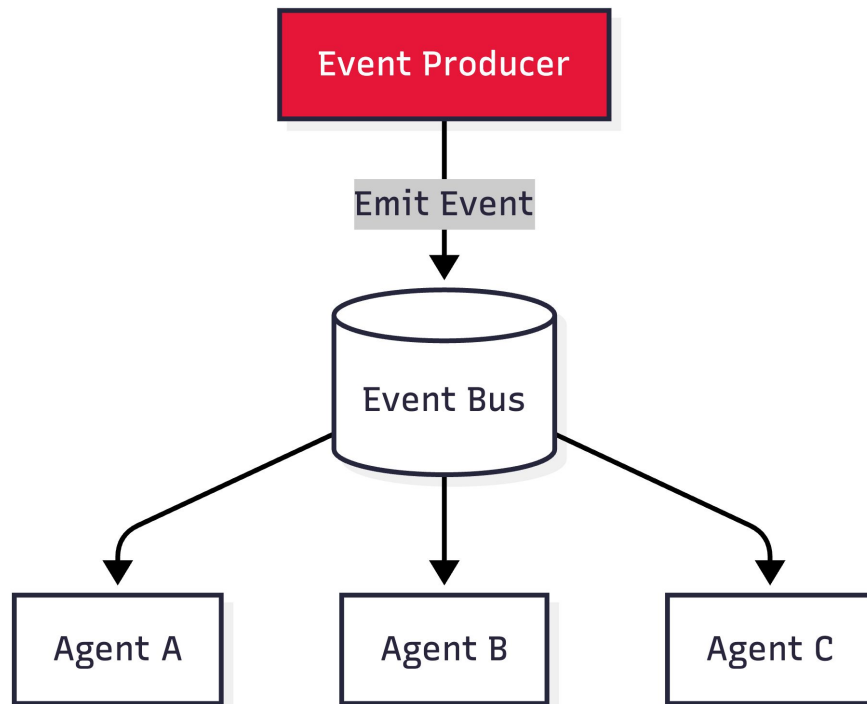
Shared Memory Approach – Where It Works Well

Some examples are:

- Knowledge aggregation
- Blackboard systems
- Long-lived context storage
- System-wide state tracking

Event-Driven Coordination

Reactive communication model: Agents subscribe to events and act only when triggered. Efficient and scalable, but requires robust event design and monitoring.



Example Event-Driven Coordination

Microservices API Calls in a distributed web application:

- Service A calls Service B via API
- Service B returns response
- Each request contains structured payload + metadata

Event-Driven Coordination: Pros & Cons

Event driven systems scale by decoupling, not by centralizing. They fail when you try to use them as a substitute for planning, validation, and termination logic.

Pros:

- Reduces polling
- Scales well with many agents
- Natural decoupling
- Reactive and efficient

Cons:

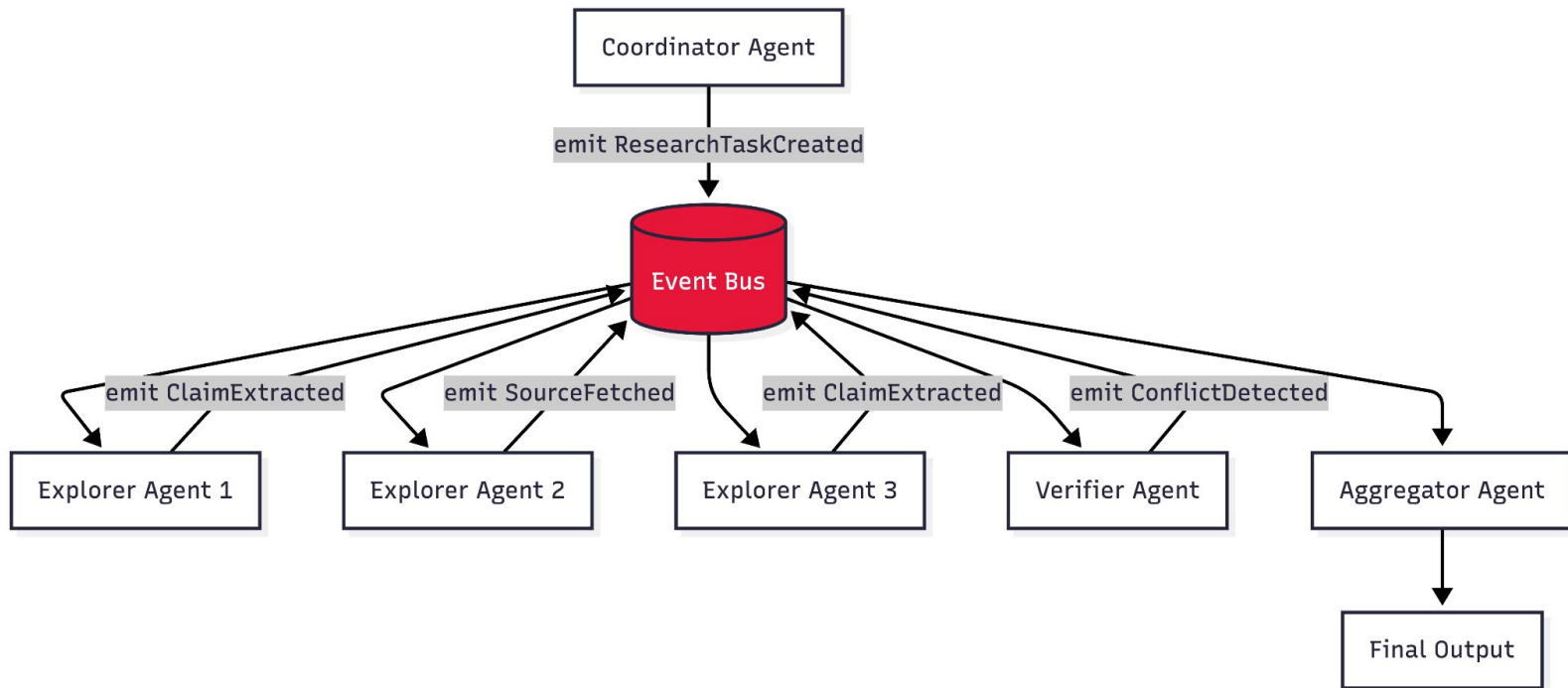
- Missed events
- Delayed propagation
- Hard-to-debug distributed reactions
- Cascading trigger loops

Event-Driven Coordination – Where It Works Well

Some examples are:

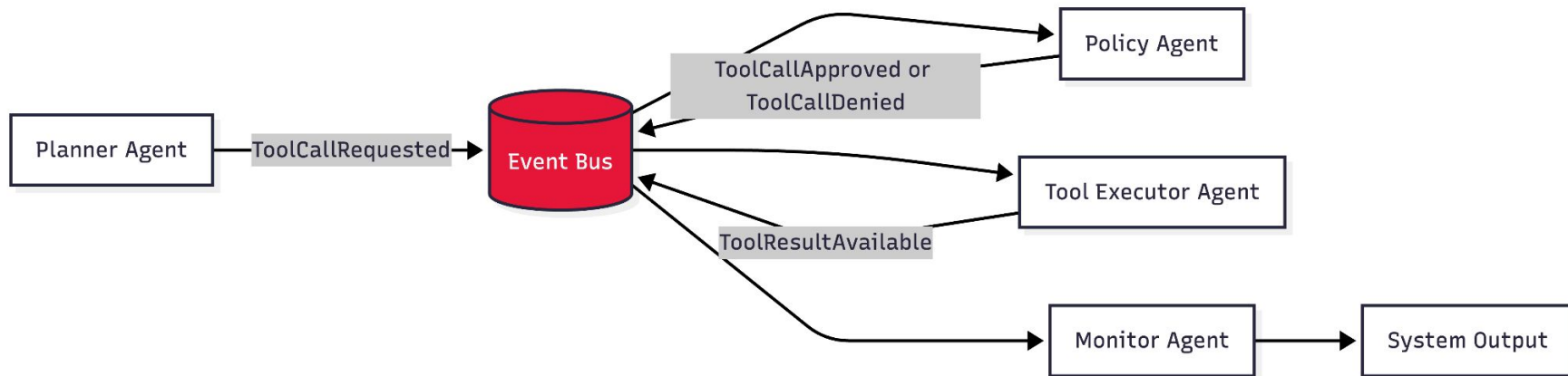
- You have many agents that should not run continuously
- You want fan out parallelism with decoupled roles
- You need asynchronous approval or escalation paths
- You want event logs as an audit trail
- You want to avoid polling and reduce idle compute

Research Task Flow



Agents wake on events, act, then publish new events. This scales well for fan out exploration and for governance checkpoints because agents remain decoupled.

Example: Event Driven Governance Gate for Tool Use



Approval, execution, and monitoring are separate agents linked by events. This prevents the planner from becoming the single control plane and creates an auditable decision trail.

Comparative Overview

Protocol	Best For	Strength	Primary Risk
Message Passing	Explicit workflows	Modularity & traceability	Latency accumulation
Shared Memory	Global state coordination	Transparency	Synchronization bottlenecks
Event-Driven	Reactive systems	Scalability	Event complexity

Choosing the Right Protocol

Ask:

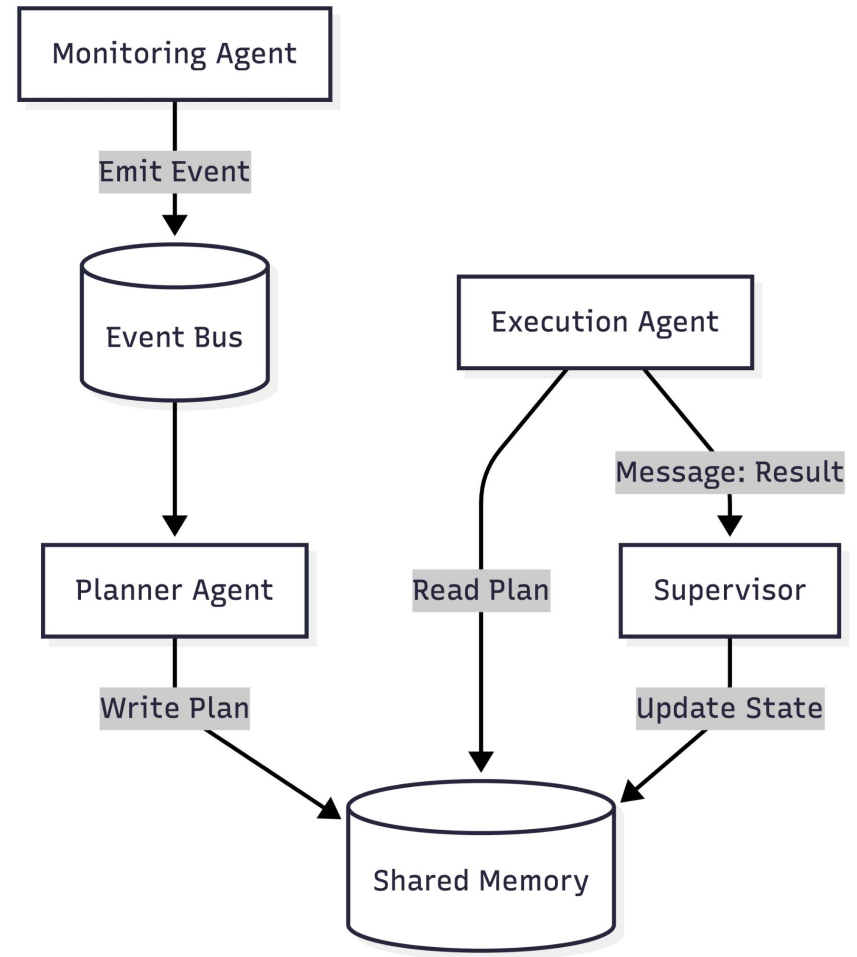
- Do agents need explicit contracts? → Message Passing
- Do they need a shared world model? → Shared Memory
- Do they react to triggers? → Event-Driven

Most production systems are hybrid.

- Swarm exploration → Event-driven
- Shared research board → Shared memory
- Final aggregation → Message passing

Hybrid Architecture

Hybrid architectures mix Event-driven triggers, shared memory state, and message passing for explicit delegation. This balances scalability, transparency, and control.



The System-Level Insight

Collaboration pattern defines topology. Communication protocol defines signal propagation. A good topology with poor communication still fails. Many systems collapse because:

- Messages weren't validated
- Memory wasn't synchronized
- Events weren't bounded





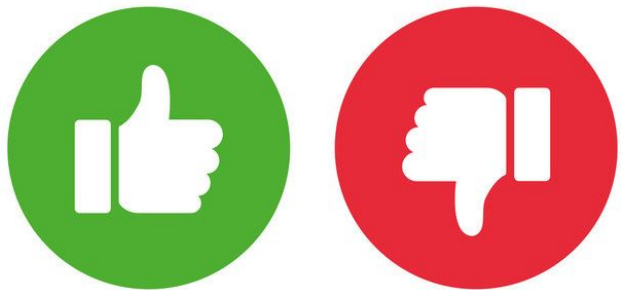
Key Takeaways Presentation Session 5

Communication patterns in multi-agent systems

- Topology defines structure
- Protocol defines signal flow
- Hybrid systems dominate production

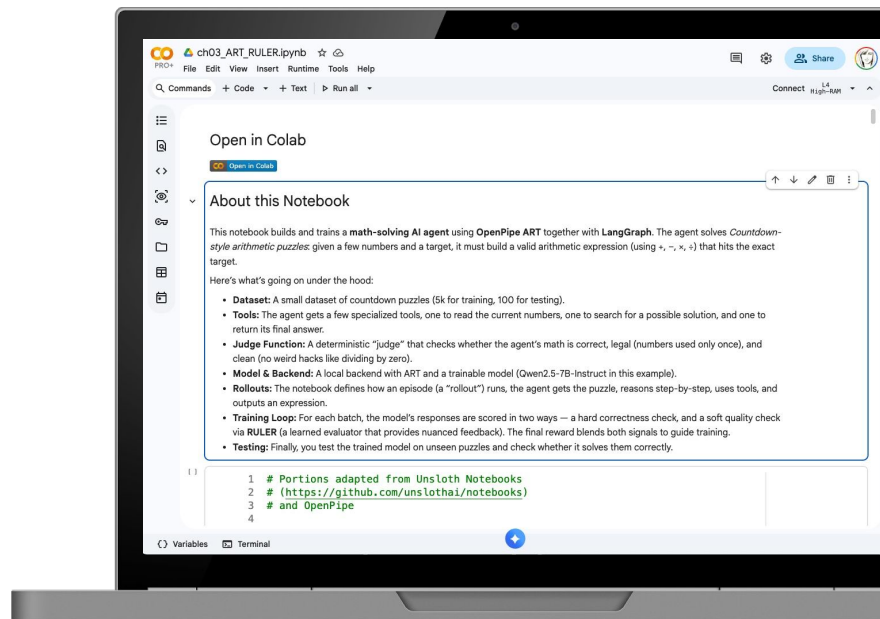
Pulse Check

Do you see how communication patterns influences your agentic system?



Demo

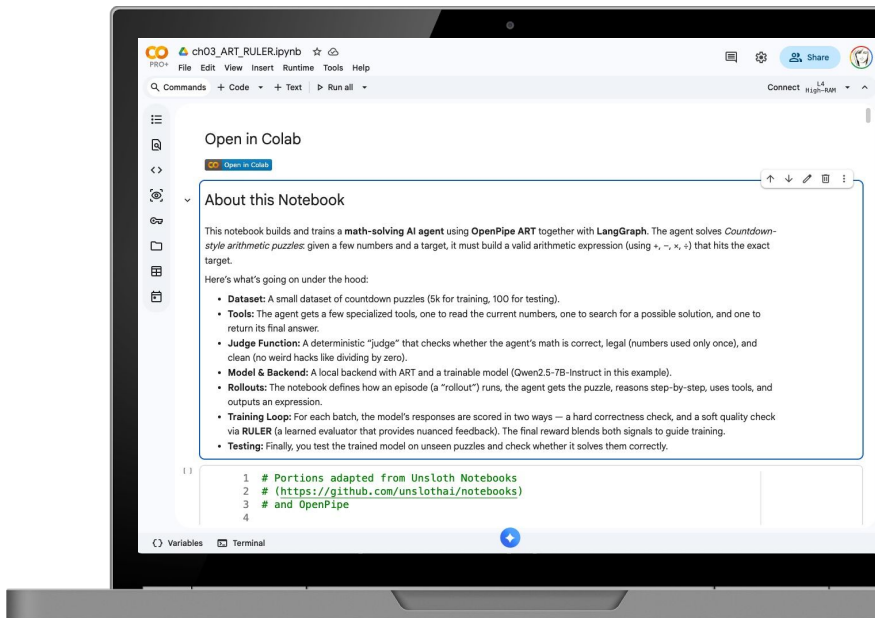
How different multi-agent collaboration patterns work together with communication patterns; which ones to combine for which use case



Hands-on

Exchange a communication pattern in a predefined MAS and observe how it affects agent coordination and communication

- Instructions are in the Colab Notebook.



Day One Key Takeaways

- Agents are compiled state machines.
- Intelligence is structured search.
- Autonomy requires authority boundaries.
- More agents introduce coordination tax.
- Communication protocol determines failure modes.
- Production systems are hybrid.

Q&A



The image features the O'Reilly logo in white text on a blue background. The background has a gradient from dark blue on the left to light blue on the right. There are two large, semi-transparent blue circles on the left side, one partially overlapping the other. The logo itself is centered horizontally and consists of the word "O'REILLY" in a bold, sans-serif font, followed by a registered trademark symbol (®).

O'REILLY®